



DATAGIA

Plataforma Digital
Agroalimentaria de
Castilla y León



INSTITUTO
TECNOLÓGICO
AGRARIO
Junta de Castilla y León
Consejería de Agricultura, Ganadería
y Desarrollo Rural

Desarrollo de un Modelo Deep Ensemble Learning para la predicción de enfermedades y plagas en viñas

Sector: Vitivinícola

16/03/2026



Financiado por
la Unión Europea
NextGenerationEU



GOBIERNO
DE ESPAÑA

MINISTERIO
PARA LA TRANSICIÓN ECOLÓGICA
Y EL RETO DEMOGRÁFICO



Plan de Recuperación,
Transformación
y Resiliencia



Junta de
Castilla y León

Título - Proyecto	Desarrollo de un Modelo Deep Ensemble Learning para la predicción de enfermedades y plagas en viñas
Acrónimo	EPVINE
Organismo	AIR Institute
Lote	2
Entregable	
Realizado por	Equipo de ciencia de datos del AIR Institute
Versión	3.0
Fecha versión	08/07/2026

REGISTRO DE CAMBIOS			
Versión	Cambio realizado	Responsable	Fecha
1.0	Versión Inicial	Equipo de ciencia de datos del AIR Institute	16/03/2026
1.1	Correcciones ajustadas al checklist	Equipo de ciencia de datos del AIR Institute	07/05/2026
1.2	Correcciones derivadas de revisión interna	Equipo de ciencia de datos del AIR Institute	01/06/2026
2.0	Correcciones derivadas del auditor	Equipo de ciencia de datos del AIR Institute	22/06/2026
3.0	Correcciones derivadas del auditor	Equipo de ciencia de datos del AIR Institute	08/07/2026

Índice

Lista de Tablas.....	4
Lista de Figuras.....	4
Listado de Abreviaturas.....	5
1 Introducción.....	5
1.1 Estado del Arte y Contexto Técnico.....	6
2 Objetivos.....	7
3 Dataset y Pipeline de Datos (ETL).....	9
3.1 Ausencia de Datos y Solución propuesta.....	9
3.2 Fundamentación Científica para el Modelado del Dataset Sintético.....	10
3.3 Variables e Ingeniería de Características.....	23
3.4 Análisis Exploratorio de Datos.....	25
3.5 Obtención del conjunto de datos.....	29
3.6 Pipeline de ETL.....	30
4 Especificaciones Técnicas.....	31
4.1 Arquitectura y Algoritmos.....	31
4.2 Entradas y Salidas.....	32
4.3 Requisitos de Hardware y Entorno.....	34
5 Análisis y Selección de Modelos.....	36
5.1 Resultados Comparativos Clasificación.....	36
5.2 Resultados Comparativos Regresión.....	37
5.3 Conclusiones de la Comparativa.....	38
6 Proceso de Entrenamiento y Evaluación.....	39
6.1 Descripción de los Procesos.....	39
6.2 Hiperparámetros y Arquitectura Clave.....	40
6.3 Estrategia de Validación y Splits.....	44
6.4 Orquestación y Criterios de Parada.....	45
7 Métricas y Rendimiento.....	46
7.1 Criterios y Métricas de Éxito.....	46
7.2 Validación cruzada y fine-tuning.....	48
7.3 Resultados en Conjunto de Test (Hold-Out).....	50
7.4 Evaluación Operativa y Visión de Negocio.....	53
8 Despliegue e Inferencia del Modelo.....	55
8.1 Modo de Despliegue.....	55
8.2 Mecanismo de Inferencia.....	55
9 Reproducibilidad y Evidencias.....	58
9.1 Inventario de Artefactos y Entorno.....	58
9.2 Pasos para Reproducir el Ciclo Completo.....	58
9.3 Trazabilidad del Código.....	60
9.4 Evidencias y Resultados.....	61
10 Limitaciones, Riesgos y Conclusiones.....	61
10.1 Desafíos y Limitaciones del Conjunto de Datos.....	61

10.2 Implicaciones Operativas en la Toma de Decisiones.....	62
10.3 Comportamiento Esperado en Despliegue.....	63
10.4 Escalabilidad y Líneas de Trabajo Futuro.....	64
10.5 Conclusiones Generales.....	64
Referencias.....	65

Lista de Tablas

Tabla 1. Listado de Abreviaturas.....	5
Tabla 2 Pipeline de ETL.....	31
Tabla 3. Tiempos de entrenamiento e inferencia con hardware de referencia.....	35
Tabla 4 Comparativa de métricas en el conjunto de Test para Clasificación.....	36
Tabla 5 Comparativa de métricas en el conjunto de Test para Regresión.....	37
Tabla 6 Métricas del DEL en Validación para varios valores del Tamaño de la Ventana de contexto.....	40
Tabla 7 Métricas del DEL en Validación para varios Escaladores.....	41
Tabla 8 Métricas del DEL en Validación para varias Técnicas de Padding.....	42
Tabla 9 Métricas del DEL en Test con y sin la variable VOC.....	42
Tabla 10. Resultados de Validación Cruzada y Fine Tuning del modelo DEL.....	49
Tabla 11. Métricas de Clasificación en el conjunto de Test del modelo DEL.....	50
Tabla 12. Métricas de Regresión en el conjunto de Test del modelo DEL.....	51

Lista de Figuras

Figura 1 Evolución del grado de infección de una serie de “Healthy”.....	13
Figura 2 Evolución del grado de infección de una serie de “Black Rot”.....	14
Figura 3 Evolución del grado de infección de una serie de “Botrytis”.....	15
Figura 4 Evolución del grado de infección de una serie de “Mildiu”.....	16
Figura 5 Evolución del grado de infección de una serie de “Oídio”.....	17
Figura 6 Evolución del grado de infección de una serie de “Esca”.....	18
Figura 7 Evolución del grado de infección de una serie de “Lobesia”.....	19
Figura 8 Evolución del grado de infección de una serie de “Altica”.....	20
Figura 9 Evolución del grado de infección de una serie de “Erinosis”.....	21
Figura 10 Evolución del grado de infección de una serie de “Empoasca”.....	22
Figura 11 Evolución del grado de infección de una serie de “Araña Roja”.....	23
Figura 12 Balanceo del conjunto de datos.....	25
Figura 13 Distribución del número de días totales por Clase.....	26
Figura 14 Distribución del pH del suelo.....	26
Figura 15 Distribución de la lluvia (mm).....	27
Figura 16 Distribución de las temperaturas.....	27
Figura 17 Matriz de Correlación (de Pearson) Global del Conjunto de Datos.....	28
Figura 18 Matriz de Confusión del modelo DEL en Test.....	52
Figura 19 Métricas por clase del modelo DEL en Test.....	52
Figura 20 Métricas del modelo DEL en Test agrupando por familias de patologías en	

Listado de Abreviaturas

Abreviatura	Descripción
DEL	Deep Ensemble Learning
1D-CNN	One-dimensional Convolutional Neural Network
LSTM	Long Short-Term Memory
BiGRU	Bidirectional Gated Recurrent Unit
CPU	Central Processing Unit
GPU	Graphics Processing Unit
ETL	Extract-Transform-Load

TABLA 1. LISTADO DE ABREVIATURAS

1 Introducción

El sector vitivinícola representa una industria de alto valor añadido cuyo éxito depende críticamente de la salud del cultivo y la calidad de la uva. Sin embargo, los viñedos son **ecosistemas altamente vulnerables a diversas plagas y enfermedades fúngicas y estructurales**. La incidencia de estos patógenos supone numerosos costes para la industria, derivados tanto de la pérdida directa de producción como de la inversión masiva en tratamientos fitosanitarios.

Históricamente, la gestión de estas amenazas se ha abordado de forma reactiva aplicando tratamientos cuando el daño ya es visible, o mediante calendarios preventivos sistemáticos que resultan ineficientes. Esta dinámica genera un impacto económico negativo y compromete la sostenibilidad del terreno. En este contexto, surge la necesidad crítica de **implementar sistemas de monitorización avanzados que permitan anticiparse a la infección**, optimizando el manejo del cultivo y promoviendo una agricultura verdaderamente sostenible.

El presente proyecto se enmarca en el área de la detección temprana de enfermedades y plagas y producción vitivinícola. La solución propuesta consiste en el diseño y desarrollo de un sistema de monitorización en tiempo real apoyado en **sensórica avanzada y algoritmos predictivos**. En la arquitectura del proyecto, se trabajará con redes de sensores IIoT (Industrial Internet of Things) distribuidas por parcelas. Estos dispositivos están encargados de capturar **series temporales ininterrumpidas de datos ambientales y fisicoquímicos** clave: temperatura (ambiental), humedad (relativa y del suelo), nivel de acidez del terreno (pH), así como la emisión de gases (CO₂) y partículas volátiles (VOCs) que actúan como bioindicadores del estrés vegetal. Toda esta telemetría es procesada por un motor analítico central.

La fitopatología en la vid no es un fenómeno lineal; la germinación de un hongo o la eclosión de una plaga dependen de la interacción compleja y temporal de múltiples factores fisicoquímicos (por ejemplo, la acumulación de horas de calor combinada con ventanas específicas de humedad foliar). Los modelos matemáticos estáticos tradicionales son incapaces de capturar estos patrones multidimensionales con precisión.

Para resolver este desafío técnico, la **Inteligencia Artificial** se presenta como la solución óptima. A través del ensamblado de múltiples arquitecturas de *Deep Learning* (como redes recurrentes y convolucionales unidimensionales), el modelo es capaz de ingerir y analizar contextos temporales de **varios días de datos fisicoquímicos** crudos. La IA no solo identifica correlaciones ocultas en las series temporales, sino que predice de forma robusta la incubación de la enfermedad antes de que existan anomalías físicas irreversibles en la planta.

1.1 Estado del Arte y Contexto Técnico

La modernización de la protección fitosanitaria en la viticultura de precisión ha estado dominada en la última década por enfoques basados en **Visión por Computador** y algoritmos de **Machine Learning tabular estático**. Sin embargo, estas metodologías, si bien han supuesto un avance, presentan limitaciones estructurales críticas para un despliegue preventivo real.

La literatura científica reciente evidencia que el uso de redes neuronales convolucionales sobre imágenes multispectrales o RGB para la identificación de patologías foliares adolece de una naturaleza fundamentalmente reactiva [27]. Para que un algoritmo visual detecte la esporulación del Mildiu o la necrosis asociada al complejo de la Yesca, el daño tisular ya debe haberse materializado de forma macroscópica [1]. En esta etapa clínica, la ventana de oportunidad para aplicar tratamientos preventivos ecológicos se ha cerrado, forzando el uso de fungicidas sistémicos o erradicantes de alto impacto ambiental. Además, los sistemas de visión enfrentan severos problemas de robustez en campo debido a la variabilidad de la iluminación y la oclusión, fallando sistemáticamente en las fases pre-sintomáticas de incubación celular [28].

Como alternativa para anticiparse a los síntomas visuales, se ha propuesto el uso de redes de sensores IoT procesados mediante algoritmos de Machine Learning clásico, tales como Random Forest o Máquinas de Vectores de Soporte (SVM). Estos modelos exigen que los datos se aplanen en promedios diarios o se restrinjan a umbrales estáticos, ignorando el orden cronológico continuo del microclima. En la biología de los patógenos, la secuencialidad es el verdadero motor epidemiológico; la interacción cronológica ininterrumpida entre la

evapotranspiración, las horas de humedad foliar y el estrés térmico es lo que determina la viabilidad de una infección fúngica o la eclosión de una plaga [29].

Para superar la reactividad ineficaz de la imagen y la ceguera temporal de la estadística clásica, este proyecto fundamenta su innovación en el uso de arquitecturas Deep Ensemble Learning (DEL) sobre **series temporales continuas**. La justificación técnica para emplear un modelado heterogéneo radica en la necesidad de combinar múltiples sesgos inductivos para obtener una mayor robustez y generalización ante las diferentes patologías.

2 Objetivos

El objetivo final del proyecto es optimizar el monitoreo de los viñedos en tiempo real mediante un modelo basado en **Deep Ensemble Learning (DEL)** que procesando **series temporales** sea capaz de actuar como un Sistema Experto. Aunque el objetivo sea realizar un modelo basado en DEL, probaremos diferentes arquitecturas basadas en árboles de decisión y redes profundas individuales para justificar el uso en producción de esta arquitectura, que será la que presente resultados más robustos y confiables.

Se pretende conseguir un diagnóstico del estado de la vid en cada momento teniendo en cuenta las patologías más comunes en los viñedos españoles [2], considerando un total de **11 estados distintos (clasificación multiclase)**:

- **Estado Sano:** Planta en perfecto estado fisiológico.
- **Plagas:**
 - **Lobesia Botrana:** Polilla que pudre los racimos.
 - **Empoasca:** Insecto verde que seca hojas.
 - **Altica:** Escarabajo saltador que agujerea hojas.
 - **Erinosis:** Ácaro que abulta las hojas.
 - **Araña Roja:** Ácaro rojizo que teje telarañas.
- **Enfermedades Fúngicas:**
 - **Botrytis:** Podredumbre gris en los racimos.
 - **Black Rot:** Pudrición negra de la uva.
 - **Oidio:** Capa de polvo blanco fúngico.
 - **Mildiu:** Hongo creando manchas de aceite.
- **Enfermedades Estructurales:**
 - **Esca:** Enfermedad mortal de la madera.

Las salidas del modelo proporcionarán:

- **Estado de la vid:** Predicción del agente patógeno específico entre las 11 clases posibles (**Clasificación**) y su grado de severidad (**Regresión**).
- **Tratamiento específico:** Emisión de un diagnóstico del estado actual acompañado del tratamiento agronómico óptimo extraído de una base de conocimiento y editable por el usuario. Esta base de conocimiento consistirá en un diccionario situado en el archivo **config/config.yaml** (editable desde este archivo). Este diccionario de tratamientos se ha basado en [2], un documento que data del 2008, por lo que es posible que hayan cambiado las reglas regulatorias de este tipo de tratamientos fitosanitarios. De este modo, se entrega el formato desde el que se podrán realizar modificaciones conforme a los criterios particulares de cada viñedo para tratar cada enfermedad/plaga o actualizaciones de la normativa de la [base de datos del Ministerio de Agricultura, Pesca y Alimentación del Gobierno de España sobre el Registro de Productos Fitosanitarios](#).

Los beneficios directos de este estudio incluyen la capacidad de aplicar tratamientos justo a tiempo y sólo donde sean necesarios, minimizando el impacto económico de las plagas en la producción y reduciendo la huella química en el ecosistema.

Para considerar validado el modelo predictivo durante sus fases de prueba y asegurar su viabilidad productiva, se establece el siguiente criterio de éxito (KPI) principal:

“Conseguir una reducción de productos fitosanitarios empleados en un 10%.”

Dado que en la fase de desarrollo y validación no se dispone del registro exacto de consumo químico histórico de las parcelas para realizar una comparativa volumétrica, este KPI de negocio debe traducirse a métricas técnicas de Inteligencia Artificial. En la viticultura tradicional, el manejo de enfermedades se rige habitualmente por tratamientos preventivos sistemáticos, asumiendo un alto coste químico y económico para asegurar la cosecha. Al integrar el modelo Deep Ensemble Learning (DEL) como sistema de soporte a la decisión, la aplicación de fitosanitarios se restringirá a las alertas generadas por el algoritmo. Por tanto, el ahorro del 10% se logrará cancelando los tratamientos rutinarios en aquellas ventanas temporales que el modelo clasifique correctamente como libres de riesgo (Verdaderos Negativos). Para garantizar que este filtrado sea económicamente rentable y agronómicamente seguro, se establecen los siguientes umbrales algorítmicos realistas:

- **Objetivo de la Clasificación del Estado:** La métrica fundamental de evaluación será el F1-Score Macro. Se selecciona esta métrica porque penaliza de forma equilibrada los Falsos Positivos (aplicaciones químicas

innecesarias que anulan el ahorro) y los Falsos Negativos (infecciones no detectadas que causan pérdidas de cultivo). Teniendo en cuenta la complejidad de discriminar entre 11 clases en entornos con alto ruido biológico y climático, impondremos un objetivo de **F1-Score Macro $\geq 0,70$** . Este umbral garantiza una capacidad predictiva robusta para filtrar una cantidad suficiente de tratamientos ciegos, haciendo que la estimación del 10% de ahorro sea conservadora y alcanzable.

- **Objetivo de la Regresión del Grado de Infección:** El objetivo se centra en minimizar el RMSE (para penalizar fuertemente las desviaciones predictivas graves) y en maximizar el coeficiente de determinación (R^2). En el modelado ecosistémico, donde la variabilidad climática natural es inherente, exigiremos un **$R^2 \geq 0,85$** . Este valor confirma que el modelo comprende la varianza subyacente de la infección sin caer en el sobreajuste (*overfitting*) propio de intentar alcanzar precisiones sintéticas del 100%, permitiendo una monitorización fiable de la severidad del patógeno.

Con todo esto, consideramos validado este estudio cuando se verifique lo siguiente:

“Desarrollar un modelo DEL que alcance las siguientes métricas:

- En clasificación multiclase (entre 11 posibles): **F1-Score Macro $\geq 0,70$** .
- En regresión del grado de infección (entre 0 y 1): **$R^2 > 0,85$** .”

En definitiva, el objetivo principal de este proyecto es el desarrollo de un sistema de Inteligencia Artificial capaz de predecir el estado de la vid a través de series temporales para conseguir detectar de forma temprana enfermedades y plagas y así poder reducir costes en relación a pérdidas en el cultivo y fitosanitarios y tratamientos empleados.

Adicionalmente, dado que la finalidad principal del sistema experto es la optimización operativa del viñedo, **la evaluación técnica se complementará con un análisis de rendimiento agrupado por familias taxonómicas de tratamiento**. Esto garantizará que, aun ante escenarios de alta redundancia climática inter-clase, el dictamen terapéutico final mantenga una precisión óptima para la toma de decisiones.

3 Dataset y Pipeline de Datos (ETL)

3.1 Ausencia de Datos y Solución propuesta

El entrenamiento de arquitecturas *Deep Learning* orientadas al análisis de series temporales exige un **volumen masivo de datos continuos**, coherentes y, sobre todo, **rigurosamente etiquetados**.

Si bien existe una abundancia de repositorios centrados en imágenes para el entrenamiento de algoritmos de Visión por Computador, **la disponibilidad de conjuntos de datos tabulares públicos basados en series temporales agronómicas es prácticamente nula**. Por otro lado, la monitorización avanzada de viñedos comerciales reales mediante redes IIoT requiere una instrumentación costosa. Los datos resultantes de estas instalaciones privadas rara vez se publican bajo licencias de código abierto debido a su alto valor comercial y estratégico.

Ante la imposibilidad de acceder a registros históricos de campo y la diversidad de variables necesarias, se determinó como requisito técnico la **generación de un dataset sintético**. La creación de un entorno de simulación propio no solo soluciona el problema de la disponibilidad de datos, sino que permite generar un ecosistema balanceado y libre de valores nulos impredecibles, proporcionando al *Deep Ensemble Learning* las condiciones necesarias para aprender de forma robusta la dinámica de incubación de las patologías.

El *script* de Python **vid_simulator.py** se encarga de replicar toda el fundamento científico de la Sección 3.2 para conseguir un **conjunto de datos sintético** que trate de parecerse lo máximo posible a la evolución o el desarrollo de una plaga/enfermedad en la vid. De este modo, para conseguir el conjunto de datos, debemos ejecutar el archivo **scripts/data/vid_simulator.py** el cual generará el fichero **data/raw/data_vin_raw.parquet**, que contendrá por defecto **989 series temporales multivariantes de evolución de diferentes patologías** (fijadas con una semilla para reproducibilidad, escogida la *random_seed* 42).

Con el propósito de garantizar la representatividad del conjunto de datos y mitigar la brecha de transferencia al entorno real (Sim-to-Real gap), el simulador descarta el uso de generadores estocásticos para las variables climáticas. En su lugar, la arquitectura se alimenta estrictamente de series meteorológicas históricas reales. De este modo, aunque la evolución de la patología se modele matemáticamente, el espacio de características (*feature space*) conserva la distribución estadística, la estacionalidad latente y las correlaciones termodinámicas genuinas del mundo físico. Esta estrategia metodológica asegura que el ensamble neuronal extraiga patrones a partir de dinámicas climáticas veraces, maximizando la robustez del modelo y aproximando de forma fidedigna las métricas de validación al rendimiento esperado durante el despliegue con sensores de campo.

3.2 Fundamentación Científica para el Modelado del Dataset Sintético

Ante la inexistencia de repositorios públicos que vinculen patologías con registros de sensores, la construcción de las series temporales tabulares se ha abordado

mediante un enfoque híbrido: **la integración de clima histórico real** sobre el que se proyecta un **modelado computacionalmente eficiente de la evolución del grado de infección (entre 0 y 1) de cada patología**. Para ello, se definieron condiciones ambientales óptimas basadas en la biología para cada una de las **11 clases** del modelo, permitiendo al sistema aprender la correlación fidedigna entre el entorno climático de la parcela y la manifestación visual de la enfermedad.

A continuación, se expone la caracterización fisicoquímica de cada una de las clases, es decir, las condiciones que se deben reunir para que se dé cada una de las patologías a detectar y la construcción de las series temporales. Fundamentalmente trataremos de **transitar desde un estado sano hasta un estado infectado mediante un modelado biológico fundamentado en la literatura**, que en esta versión interactúa dinámicamente con ventanas climáticas reales. Como el comportamiento de cada enfermedad es diferente, expondremos una por una cuál ha sido el criterio escogido para cada representación.

Todas y cada una de estas patologías están representadas y estudiadas a fondo en el *notebook* **EDA_Data_Vin.ipynb**.

Simulación de variables fisicoquímicas:

- **Obtención del clima histórico y fenología:** Para garantizar el máximo rigor agronómico, la base meteorológica descarta el uso de generadores estocásticos y se alimenta directamente de datos climáticos históricos reales obtenidos a través de la [API de Open-Meteo \(Historical Weather API\)](#). Se han extraído registros horarios continuos desde el 1 de enero de 2015 hasta el 31 de diciembre de 2025 para **5 parcelas** o estaciones representativas de las principales Denominaciones de Origen (D.O.) vitivinícolas **de Castilla y León**:
 - Roa de Duero (D.O. Ribera del Duero): Latitud 41.69, Longitud -3.93, Elevación 750 m.
 - Rueda (D.O. Rueda): Latitud 41.41, Longitud -4.96, Elevación 720 m.
 - Toro (D.O. Toro): Latitud 41.52, Longitud -5.39, Elevación 620 m.
 - Cacabelos (D.O. Bierzo): Latitud 42.60, Longitud -6.73, Elevación 480 m.
 - Cigales (D.O. Cigales): Latitud 41.76, Longitud -4.70, Elevación 725 m.

De esta API se extraen de forma directa las variables de **temperatura a 2m, humedad relativa a 2m, precipitación, velocidad del viento a 10m y**

radiación solar de onda corta. La obtención de estas variables se realiza a partir del *script* **`scripts/data/descarga_clima_historico.py`**. A partir de estas, se calcula la Evapotranspiración de Referencia (ET_0). Para asegurar la viabilidad temporal, el simulador selecciona aleatoriamente bloques de este histórico acordes con los meses biológicamente activos de cada patología, validando que las primeras 24 horas presenten condiciones sanas (sin eventos extremos). Respecto a la fenología, el modelo computacional sustituye la clásica curva escalonada del coeficiente de cultivo (K_c) estipulada en la metodología FAO-56 por una función logística generalizada continua. Esta adaptación agronómica escala progresivamente la demanda evaporativa ($ET_c = ET_0 K_c$), garantizando transiciones suaves y derivables, un requisito indispensable para optimizar el aprendizaje de las redes neuronales subyacentes del Gemelo Digital [3].

- Modelo dinámico de condiciones de suelo:** La simulación de la **humedad edáfica** se fundamenta en un balance hídrico continuo de capa única [4]. El volumen de control se encuentra físicamente acotado por la Capacidad de Campo (CC) y el Punto de Marchitez Permanente (PMP), empleando parámetros hidrodinámicos estándar para suelos franco-arcillosos [5]. El algoritmo calcula el agotamiento diario restando la evapotranspiración del cultivo (ET_c) y sumando la precipitación efectiva. De manera crítica, el modelo incorpora la respuesta fisiológica de la vid al estrés hídrico: cuando el nivel de agua desciende por debajo de un umbral de seguridad (agotamiento superior al 60% del agua útil), el código aplica un coeficiente de estrés dinámico (K_s) de decaimiento lineal. Este coeficiente reduce la evapotranspiración real de la planta, simulando el cierre estomático y garantizando la coherencia del Gemelo Digital.
- Gases de la Planta (CO_2 y VOC):** La simulación de los intercambios gaseosos (CO_2 y Compuestos Orgánicos Volátiles, VOC) se fundamenta en un modelo biofísico acoplado a las condiciones meteorológicas y al estado sistémico de la vid. La concentración de CO_2 base (410 ppm) es modulada por un ciclo diurno determinista: durante el día, la eficiencia fotosintética —penalizada proporcionalmente al daño foliar acumulado— actúa como sumidero, mientras que por la noche, la respiración celular, exacerbada por el estrés biótico, actúa como fuente de emisión. Por su parte, la dinámica de los VOC integra un ruido de fondo por estrés abiótico dependiente de la temperatura y una respuesta inmunitaria biótica. Esta última se disocia por completo de las etiquetas categóricas de la etiología; en su lugar, se cuantifica a partir de la inercia del daño tisular activo. Matemáticamente, esto se implementa aplicando una Media Móvil Exponencial (EMA) sobre la

derivada discreta del grado de infección con respecto al tiempo, $\Delta I/\Delta t$. Las concentraciones locales de ambos gases son posteriormente atenuadas mediante funciones de dispersión eólica y lavado pluvial, añadiendo finalmente ruido blanco gaussiano para emular las desviaciones de calibración e incertidumbre inherentes a los sensores de campo.

Simulación del estado sano:

- **Sano (Control):** Representa la línea base fisiológica de asimilación de CO₂ [6] y emisiones basales mínimas dependientes de la temperatura y la luz [7], sin cruzar umbrales de perturbación. Se puede apreciar la evolución de una trayectoria de estado sano en la Figura 1.

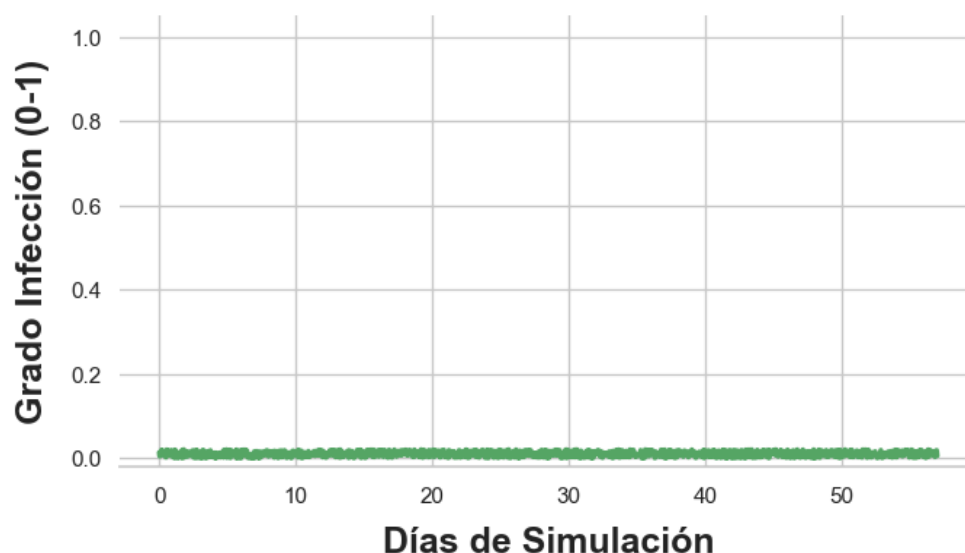


Figura 1 Evolución del grado de infección de una serie de "Healthy"

Simulación del desarrollo de enfermedades en la vid:

- **Black Rot (*Guignardia bidwellii*):** El modelo de infección se basa en los estudios de Spotts [8], los cuales demuestran que el éxito de la penetración fúngica depende de una relación no lineal entre la temperatura y la duración del mojado foliar. En el simulador, la tabla de requerimientos de Spotts se ha traducido a una **función polinómica** acotada entre los 7°C y los 32°C. Esta curva establece dinámicamente el umbral mínimo de horas de humedad necesarias para incrementar la infección, alcanzando su óptimo térmico (mínima exigencia de mojado, acotada estrictamente a 6 horas) en el rango de los 21°C a 25°C. Una vez superado el umbral diario calculado, el modelo acumula severidad de forma proporcional al exceso de

horas de mojado, simulando el incremento en la tasa de éxito germinativo de las ascosporas. Se puede apreciar la evolución de una trayectoria de Black Rot en la Figura 2.



Figura 2 Evolución del grado de infección de una serie de "Black Rot"

- Botrytis (*Botrytis cinerea*):** El riesgo de infección se modela implementando la **ecuación Logit** [9], la cual determina la probabilidad de colonización del hongo en función de la interacción no lineal entre la temperatura y las horas continuas de mojado foliar. De acuerdo con la literatura base, el algoritmo desencadena el cálculo de riesgo superadas las 4 horas de humedad, alcanzando su probabilidad máxima en la ventana térmica óptima de los 15°C a 22°C. Adicionalmente, el simulador incorpora un multiplicador de severidad dinámico condicionado a la existencia de daño mecánico previo (como los daños causados por Lobesia Botrana). Esta aproximación computacional refleja fielmente el marco de vías de infección secundarias y la interacción patógeno-huésped descrito en [10], modelando una perturbación biótica crítica que impacta de forma directa sobre las emisiones de volátiles (VOCs) basales de la vid. Se puede apreciar la evolución de una trayectoria de Botrytis en la Figura 3.

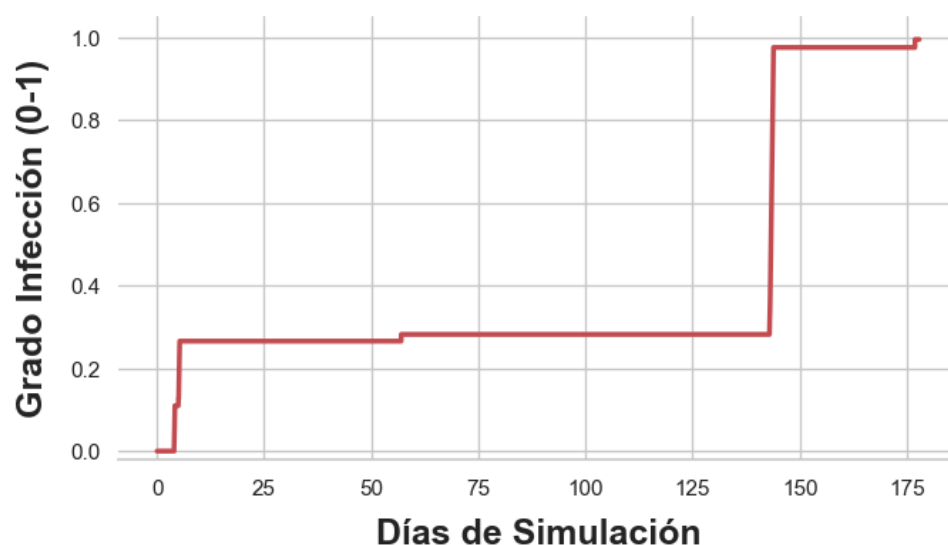


Figura 3 Evolución del grado de infección de una serie de "Botrytis"

- Mildiu (*Plasmopara viticola*):** La dinámica de la infección primaria se ha modelado siguiendo los **criterios empíricos clásicos de los "3-Dieces"**, que vinculan el inicio de la actividad fitopatógena con brotes superiores a 10 cm, temperaturas medias $>10^{\circ}\text{C}$ y precipitaciones >10 mm en 24 horas. Para la fase de latencia, el algoritmo implementa la lógica de [11], calculando la duración del periodo de incubación como una función hiperbólica de la temperatura efectiva. La manifestación visual de los síntomas está condicionada en el modelo por un requisito biofísico de oscuridad y humedad relativa superior al 92%, reflejando el comportamiento estomático del patógeno descrito en modelos mecanicistas de referencia [12]. Se puede apreciar la evolución de una trayectoria de Mildiu en la Figura 4.



Figura 4 Evolución del grado de infección de una serie de “Mildiu”

- Oídio (*Erysiphe necator*):** Aunque la literatura fitopatológica establece que el ciclo biológico del patógeno se inicia con la liberación primaveral de ascosporas [13] y queda limitado tardíamente por la resistencia ontogénica de las bayas [14], la simulación de su fase epidémica secundaria (crítica para el cálculo del estrés continuo) se rige estrictamente por un **modelo térmico**. Para ello, el algoritmo implementa una versión parametrizada del Índice de Riesgo de Gubler-Thomas (GTI) [15]. Este índice evalúa diariamente la acumulación de horas en el rango térmico óptimo (21-30°C) para aumentar la presión de la enfermedad. Adicionalmente, el modelo aplica **penalizaciones severas por temperaturas letales (>35°C)** y eventos de precipitación. La puntuación dinámica resultante (0-100) se mapea para determinar la cobertura del desarrollo epidérmico, simulando el incremento sostenido de la respiración celular superficial en la planta. Se puede apreciar la evolución de una trayectoria de Oídio en la Figura 5.



Figura 5 Evolución del grado de infección de una serie de “Oídio”

- Yesca (*Esca*):** La dinámica de este patógeno vascular se ha modelizado computacionalmente mediante un avanzado proceso estocástico de difusión con saltos (Jump-Diffusion), estructurado para capturar la naturaleza dual de la enfermedad descrita en la literatura [16, 17]. Por un lado, la fase crónica se simula mediante una degradación lineal. Por otro lado, la fase aguda se aborda bajo la hipótesis del fallo vascular provocado por un desequilibrio entre la demanda atmosférica (ET_0) y la oferta hídrica [18]. Cuando este Índice de Estrés Hidráulico (HSI) supera un umbral crítico bajo condiciones térmicas estivales ($>25^{\circ}\text{C}$), el algoritmo detona un proceso de Poisson cuya probabilidad de ocurrencia crece exponencialmente, simulando la cavitación catastrófica y el colapso súbito y definitivo de la cepa. Se puede apreciar la evolución de una trayectoria de Yesca en la Figura 6.

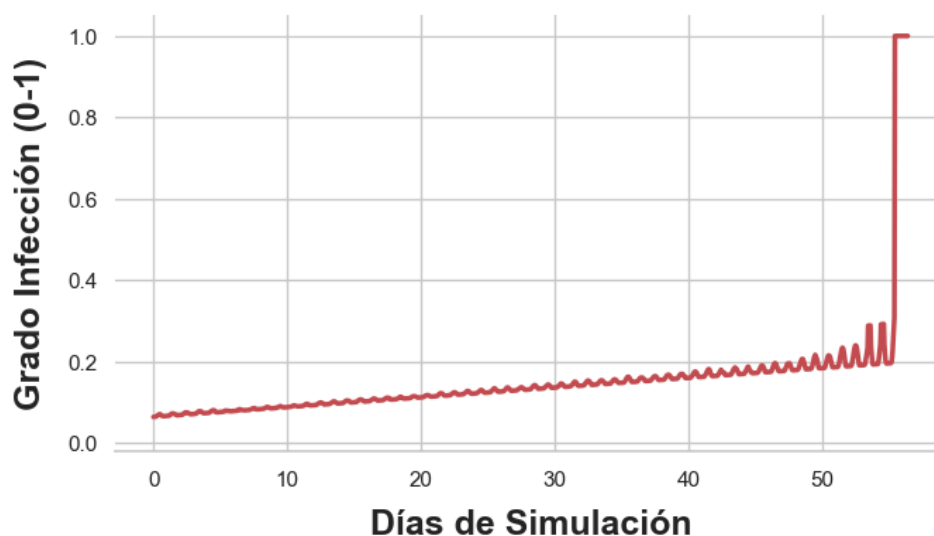


Figura 6 Evolución del grado de infección de una serie de “Esca”

Simulación del desarrollo de plagas en la vid:

- Polilla del Racimo (*Lobesia Botrana*):** La simulación de la dinámica poblacional de *Lobesia botrana* implementada en el Gemelo Digital se fundamenta en un **modelo fenológico empírico regido por la acumulación de la integral térmica (Grados-Día o GDD)**. Este enfoque algorítmico permite pronosticar de forma robusta y computacionalmente eficiente los picos de actividad de vuelo de las sucesivas generaciones de la polilla, una metodología ampliamente validada por la literatura agrometeorológica para establecer umbrales de desarrollo [19]. La predicción precisa de estos escenarios, estructurada en el código mediante superposición de ondas gaussianas, es una herramienta crítica a nivel agronómico para anticipar el daño en el racimo y programar intervenciones, tal y como evidencian los protocolos de seguimiento de curva de vuelo en regiones vitivinícolas de referencia [20]. Se reconoce que el estado del arte en la modelización de esta plaga ha evolucionado hacia enfoques matemáticos de alta complejidad, tales como los modelos demográficos mecanicistas basados en ecuaciones diferenciales estocásticas [21] o los recientes modelos de Regresión Lineal Múltiple (MLR) diseñados para mitigar los sesgos del cambio climático integrando variables como la radiación global o la humedad [22]. Sin embargo, el modelo de integral térmica desarrollado (enriquecido con penalizaciones climáticas directas como el lavado por precipitación) resulta ser la estrategia óptima y más coherente para la generación masiva de series temporales sintéticas,

garantizando que la red neuronal aprenda la estacionalidad base de la plaga sin incurrir en una sobrecarga computacional innecesaria. De este modo, dependiendo de la generación que intervenga de esta plaga, se tendrá un riesgo determinado, siendo el máximo el de puesta y eclosión en la 3ª generación, provocando daños severos en la baya. En la Figura 7 se observa el desarrollo del grado de infección de una trayectoria de Lobesia.

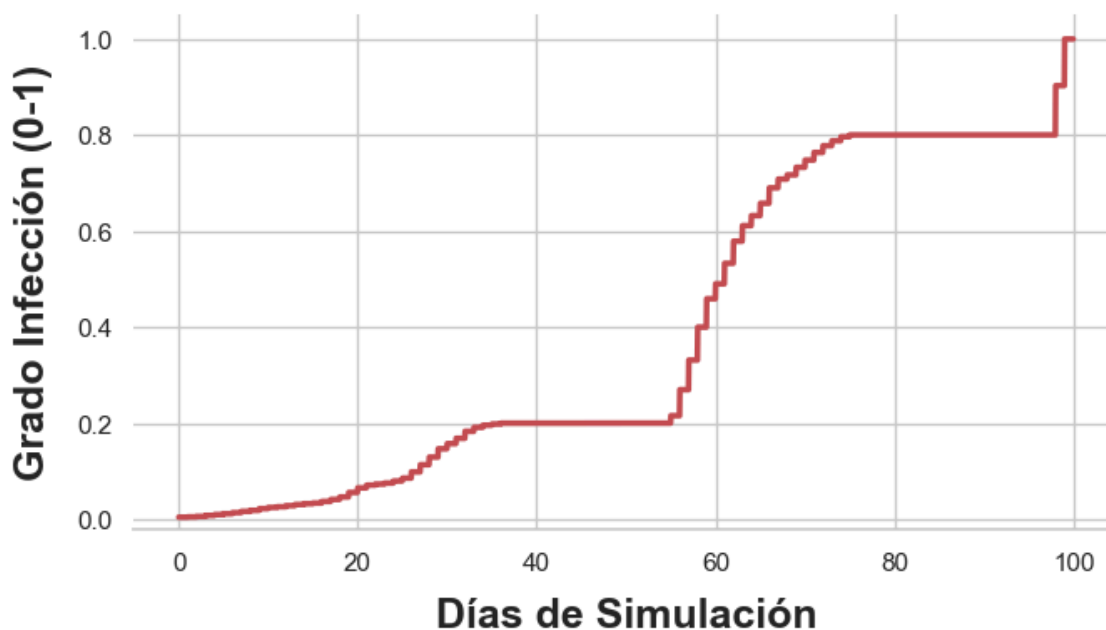


Figura 7 Evolución del grado de infección de una serie de "Lobesia"

- Altica (*Altica ampelophaga*):** A diferencia de otras plagas principales, no existen modelos matemáticos paramétricos estandarizados exclusivos para su dinámica poblacional. Aunque en la literatura general de artrópodos se proponen modelos no lineales complejos para describir el desarrollo en función de umbrales térmicos [23], por motivos de eficiencia computacional en la simulación de series sintéticas se ha optado por una parametrización heurística. Esta aproximación traduce computacionalmente la biología descrita en manuales agronómicos regionales [24] mediante un **modelo de Grados-Día (GDD)**. Dado que los adultos invernantes inician su ataque coincidiendo con el "inicio de brotación de la cepa", el algoritmo dispara la emergencia mediante una **función escalón al alcanzar un umbral de 280 GDD (base 10°C)**, un valor alineado con el acumulado térmico típico de este estadio fenológico en la vid [19]. A partir de esto, las formas móviles comienzan a alimentarse vorazmente (esqueletización foliar), lo que penaliza la asimilación de CO₂ de la planta. Para replicar con precisión el comportamiento de estos poiquiloterms en primavera, el código modula el daño activando la tasa de ingesta sólo cuando la temperatura supera los

15°C, logrando así una simulación mecanicista robusta y coherente con los síntomas observados en campo [2]. Se puede apreciar la evolución de una trayectoria de Altica en la Figura 8.



Figura 8 Evolución del grado de infección de una serie de "Altica"

- Erinosis (*Colomerus vitis*):** La infección se restringe estrictamente a la ventana fenológica de brotación, momento en el que los ácaros migran desde las yemas hacia las hojas nuevas. Tomando como base los estudios de dinámica poblacional de ácaros en distintas variedades de vid [25], el riesgo de formación de agallas se ha parametrizado heurísticamente para alcanzar su máximo **bajo condiciones de alta humedad relativa (>65%) y temperaturas suaves (18-27°C)**. Computacionalmente, esto se traduce en un incremento acumulativo del daño que permanece constante el resto de la campaña, simulando un evento de estrés biótico crónico (saturado al 50% de severidad) que altera estructuralmente la hoja y sus emisiones basales. Se puede apreciar la evolución de una trayectoria de Erinosis en la Figura 9.



Figura 9 Evolución del grado de infección de una serie de “Erinosis”

- Mosquito Verde (*Empoasca vitis*):** Su dinámica poblacional se ha modelado mediante una **ecuación de crecimiento logístico**, donde variables agronómicas y climáticas determinan la evolución de la plaga. A nivel espacial, estudios empíricos demuestran que las poblaciones de *Empoasca vitis* presentan una distribución agregada dentro del viñedo, fuertemente vinculada a las características de la planta que favorecen la oviposición [26]. Computacionalmente, esto se ha simulado haciendo que la capacidad de carga del sistema (K) sea directamente proporcional al coeficiente de cultivo dinámico (K_c), de modo que las zonas de mayor biomasa y vigor foliar soportan mayores niveles de infestación. A nivel climático, se han establecido reglas heurísticas basadas en la biología de los cicadélidos: la tasa de desarrollo y eclosión se maximiza en un rango térmico óptimo de entre 20°C y 32°C, mientras que la población sufre una fuerte penalización por desecación (aumento de la mortalidad) cuando la humedad relativa ambiental cae por debajo del 20%. Se puede apreciar la evolución de una trayectoria de *Empoasca* en la Figura 10.

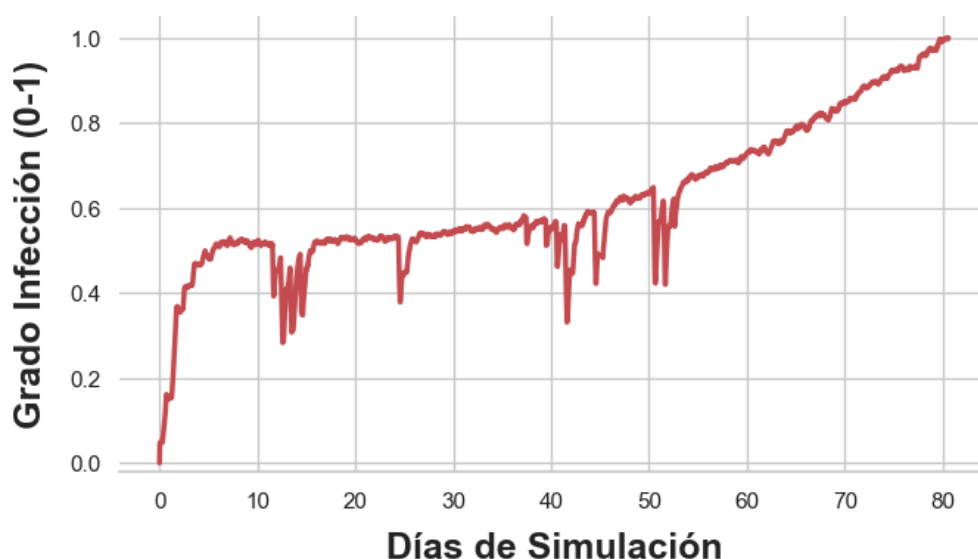


Figura 10 Evolución del grado de infección de una serie de “Empoasca”

- Araña Roja (*Tetranychidae*):** La dinámica poblacional de este ácaro experimenta un crecimiento exponencial explosivo bajo condiciones específicas de calor y sequedad. El modelo implementado fundamenta su tasa intrínseca de crecimiento (r_m) en el potencial térmico, estableciendo un umbral basal a partir del cual la tasa de desarrollo se incrementa linealmente, en estricta concordancia con los modelos de influencia térmica descritos en la literatura para esta plaga [26]. Para traducir computacionalmente el requerimiento de “calor seco” ($T > 30^\circ\text{C}$ y $\text{HR} < 40\%$), dicho potencial térmico se acopla a un factor de sequedad cuadrático que penaliza severamente la reproducción en ambientes húmedos. Adicionalmente, el algoritmo incorpora una regla heurística de control biológico y físico mediante mortalidad catastrófica por lavado pluvial, reduciendo drásticamente la población durante eventos de lluvia intensa, y limitando finalmente la infección mediante la saturación de la capacidad estomática y foliar de la planta. Se puede apreciar la evolución de una trayectoria de Araña Roja en la Figura 11.



Figura 11 Evolución del grado de infección de una serie de “Araña Roja”

Toda esta lógica se trata de replicar en el archivo de creación del conjunto de datos **scripts/data/vid_simulator.py** el cual nos permite crear el número de series que se desee de forma que se balancean automáticamente para cada enfermedad. Sin embargo, cada propagación e infección tiene una duración diferente, por lo que aunque el número de series asociadas a cada estado de la vid sea el mismo, el número de registros de cada estado no.

3.3 Variables e Ingeniería de Características

Las variables utilizadas en este proyecto son las siguientes:

- **Variables de sensores y/o meteorológicas (predictores crudos):**
 - **pH_Suelo:** Acidez del suelo, (Sensor, adimensional).
 - **CO2_ppm:** Concentración de CO2 (Sensor, ppm).
 - **VOC_ppb:** Componentes Volátiles Orgánicos de las plantas (Sensor, ppb).
 - **Temp_Amb_C:** Temperatura del aire (Sensor/Meteo, °C).
 - **Hum_Rel_Pct:** Humedad relativa del aire (Sensor/Meteo, %).
 - **Lluvia_mm:** Precipitación horaria (Sensor/Meteo, mm).
 - **Viento_kmh:** Velocidad del viento (Sensor/Meteo, km/h).
 - **Hum_Suelo_Pct:** Humedad del suelo (Sensor/Meteo, %).

- **Variables derivadas mediante ingeniería de características:**
 - **Hora:** No se utiliza como feature del modelo, solo se emplea para establecer un orden y para calcular variables cíclicas derivadas (seno y coseno).
 - **Hora_Sin/Hora_Cos:** Seno y Coseno de la hora para que la red capture la estacionalidad.
 - **Horas_Humedad_Foliar:** Valor entero acumulado que indica la cantidad de horas que la superficie está mojada. Se calcula a partir de las variables Lluvia_mm y Hum_Rel_Pct.
 - **GDD_Acumulado:** Suma acumulada de calor útil (°C). Se calcula a partir de la variable Temp_Amb_C.
- **Variables de control y metadatos (no predictoras, excluidas del entrenamiento):**
 - **Fecha:** (Timestamp) Fecha y hora exacta de la toma de datos con **granularidad horaria**. Se escogió esta resolución debido a que es similar a la de la API meteorológica, ayuda a representar y entender mejor los datos y es un valor recogido en estudios como [30]. Se necesita para inferencia.
 - **ID_Serie:** Identificador único de la serie temporal (Opcional pero recomendada para inferencia al trabajar con varias series a la vez).
 - **Parcela_ID:** Nombre de la parcela de la que se extraen los datos climáticos (Opcional: Solo se ha utilizado para la fase de extracción y experimentación).
 - **Etiqueta_Clase:** Indica la patología a la que está asociada la serie. No dispondremos de ella en producción, nos ayuda a establecer coherencia durante el desarrollo (**No se necesita para inferencia**).
- **Variables objetivo (Targets) del modelo:**
 - **Clase_Entrenamiento:** Estado actual de la vid, que se codifica con un encoder en la variable "**Etiqueta_Num**" (**Target de clasificación**).
 - **Grado_Infeccion:** Magnitud de la infección, nivel de daño real. Es un número real entre 0 y 1 (**Target de Regresión**).

3.4 Análisis Exploratorio de Datos

El análisis exploratorio del *dataset* nos permite encontrar posibles patrones y relaciones entre los datos, y a su vez, verificar que la creación del dataset sintético es correcta. Se realizó un análisis en el *notebook* [notebooks/EDA/EDA_Data_Vin.ipynb](#). Dividimos las conclusiones en los diferentes puntos:

- **Datos faltantes, duplicados y nulos:** Al tratarse de un entorno controlado alimentado por APIs climáticas depuradas y un simulador mecanicista determinista, el dataset final (constituido por 23 columnas y un volumen neto de 1.628.661 registros horarios) presenta una tasa de valores nulos o duplicados del 0.00%, garantizando la integridad de las matrices de entrada.
- **Balance de clases:** El número de series (ID_SERIE) de cada enfermedad o plaga está equilibrado. Mediante el *script*, se generaron 1.000 series temporales diferentes, de las cuales únicamente **989** son válidas.



Figura 12 Balanceo del conjunto de datos

Aún así hay que tener en cuenta que cada enfermedad o plaga se desarrolla durante un intervalo de tiempo diferente, por lo que el número total de registros de cada estado no será el mismo. Esto genera un desbalance (en cuanto al número de registros) hacia las clases que se propagan de forma más lenta. Esto se puede apreciar en la Figura 13.

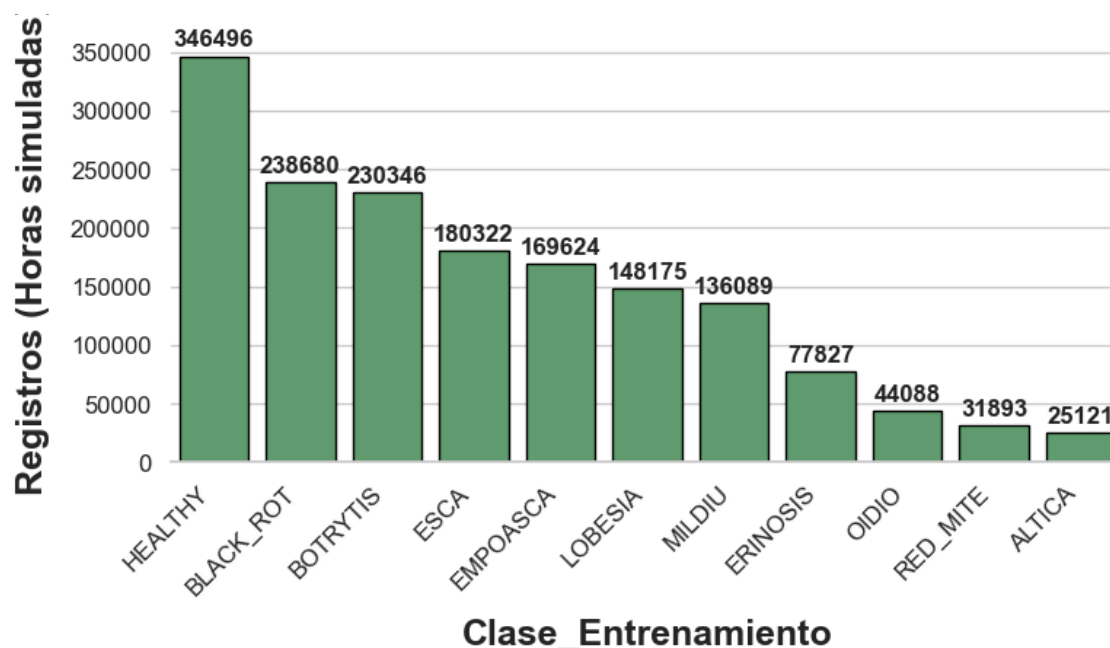


Figura 13 Distribución del número de días totales por Clase

- Análisis distribucional:** Se graficó el histograma de cada variable numérica, comprobando comportamientos lógicos como que el pH del suelo siga una distribución normal centrada en 6,8 (Figura 14), que la lluvia tenga una cola larga a la derecha (Figura 15) o que la temperatura ambiente se encuentra centrada entre 10 y 20°C (Figura 16).

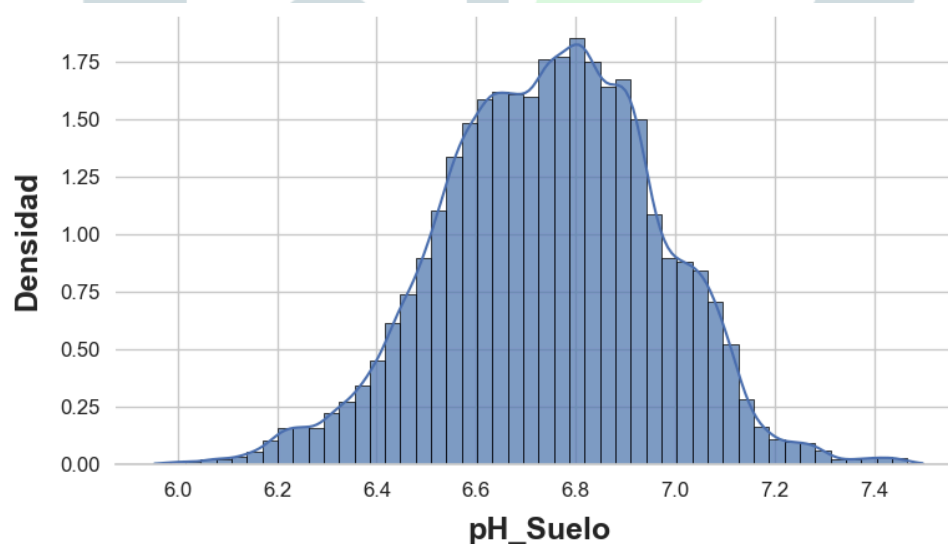


Figura 14 Distribución del pH del suelo

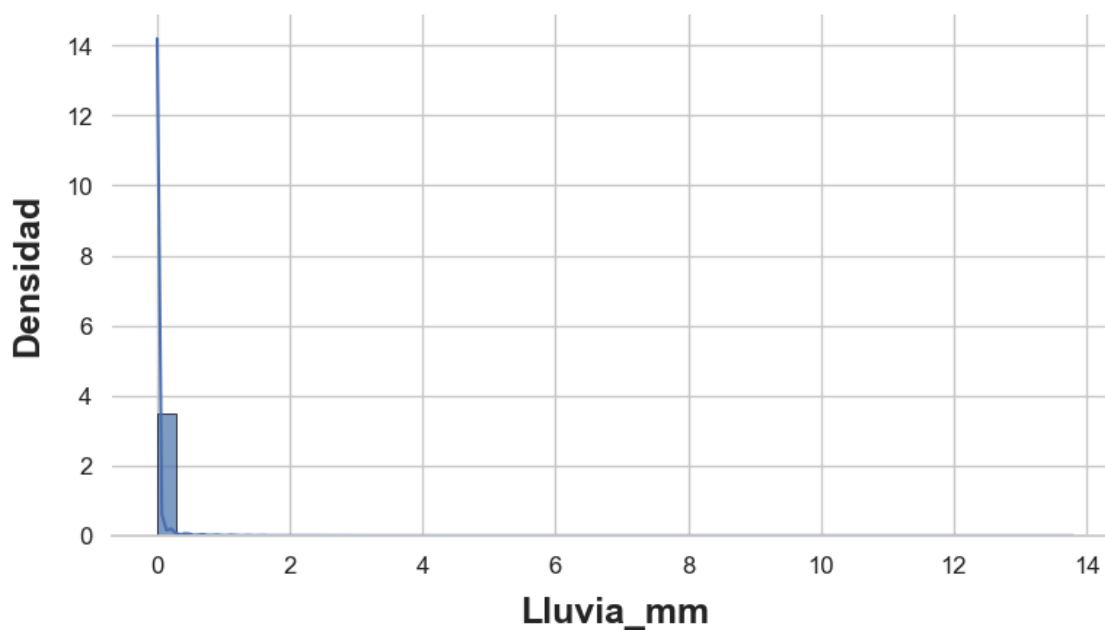


Figura 15 Distribución de la lluvia (mm)

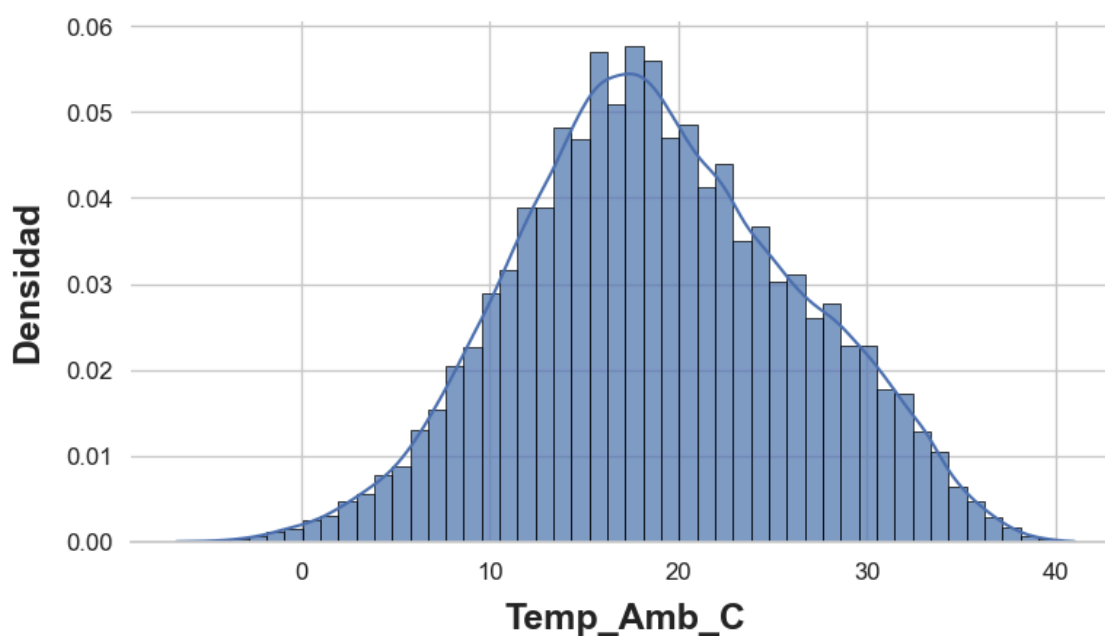


Figura 16 Distribución de las temperaturas

- **Análisis de correlaciones:** Se realizó un análisis de correlaciones para encontrar predictores altamente correlados con la variable objetivo *Grado_Infeccion* mediante la matriz de correlación de Pearson representada en la Figura 17.

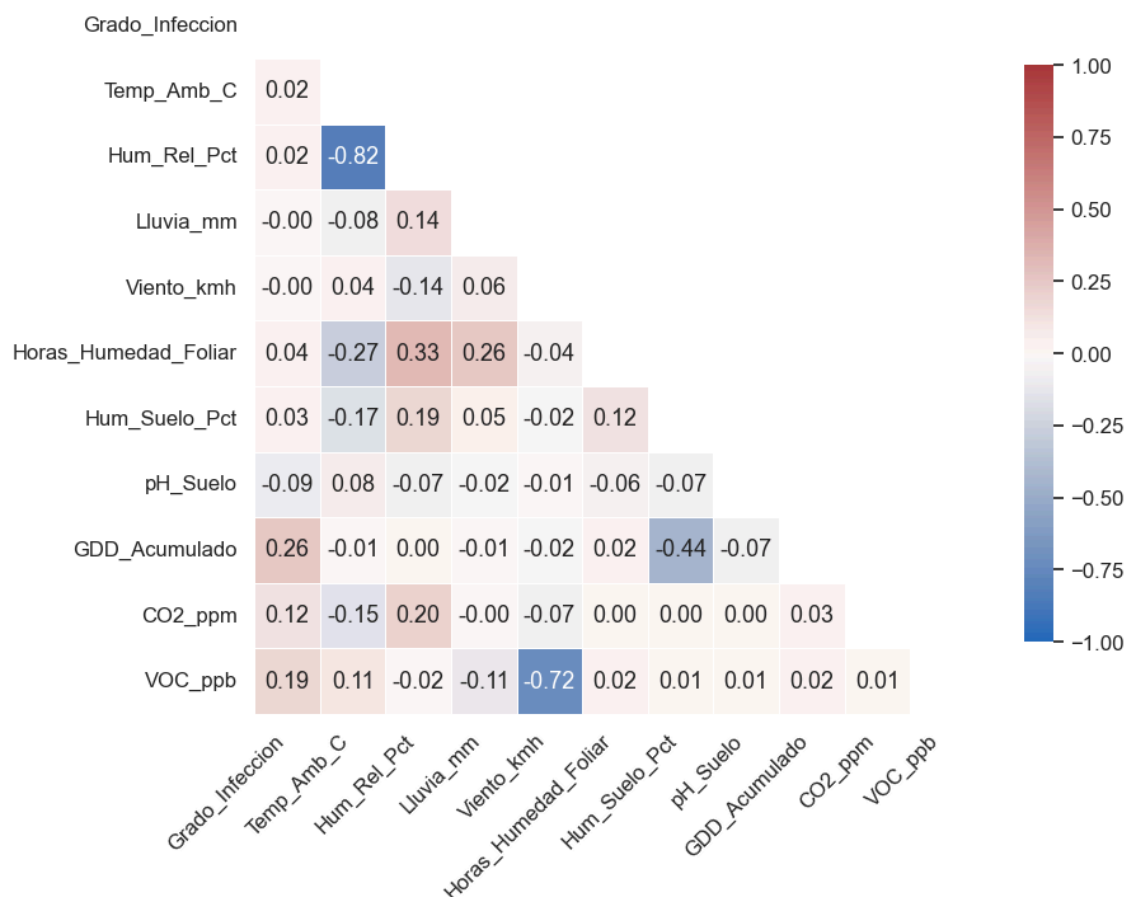


Figura 17 Matriz de Correlación (de Pearson) Global del Conjunto de Datos

Se puede observar que no hay variables muy correladas con el target (al menos linealmente). La variable con mayor poder predictivo son el GDD, el CO2 y el VOC. Sin embargo, a nivel teórico, el resto de variables sí que aportan información acerca de la propagación de enfermedades y plagas, por lo que conservaremos todas estas variables para entrenar al modelo.

- Estudio de Fuga de Datos (Data Leakage):** Dado el origen sintético del conjunto de datos y el poder predictivo observado en la sensorica de gases (CO2 y VOC), se evaluó la posible existencia de fuga de datos. Mediante un análisis de varianza (ANOVA), se estudió la influencia de ambas variables en el grado de infección y la clase de la patología. A nivel de registro horario individual, el estrés térmico de fondo y la incertidumbre del sensor enmascaran la señal. Sin embargo, al agregar la información en ventanas temporales (ej. 168 horas), emerge una clara separabilidad estadística. Esto se debe a que la separabilidad proviene de una señal predictiva biológica lícita fundamentada en la derivada del daño tisular. Las plagas (dinámica discreta) generan picos abruptos de emisión, mientras que los hongos (dinámica continua) trazan rampas suaves. Esto garantiza que las métricas

del modelo evaluarán el aprendizaje de patrones morfológicos temporales reales y no la memorización de variables espurias.

- **Dimensionalidad:** Contamos con un total de 23 columnas (en el dataset procesado pero solo 12 serán predictores) y **989 series temporales multivariantes**, formando un total de **1.628.661 filas**. Este mismo conjunto se utiliza (con el mismo particionado) para realizar la optimización de hiperparámetros, manteniendo el conjunto de test ciego para el modelo en todo momento.
- **Simulación de las trayectorias:** Se graficaron varias trayectorias de cada plaga y enfermedad para poder analizar su evolución. En estas trayectorias se comprueban fundamentos teóricos como que la *Lobesia Botrana* tiene tres generaciones o que la *Yesca* funciona por colapso, entre otras (ver Sección 3.2).

3.5 Obtención del conjunto de datos

Para obtener el conjunto de datos sintéticos sobre el que se ejecutará nuestro experimento es necesario descargar los datos meteorológicos históricos de la [API Open-Meteo](#) en las diferentes parcelas de Castilla y León mediante el *script* **`scripts/data/descarga_clima_historico.py`** y su limpieza con **`scripts/data/data_cleaner.py`**. Cuando se obtengan los datos de estos *scripts*, se debe ejecutar el script **`scripts/data/vid_simulator.py`** el cual, contiene como parámetros configurables el número de series que se van a simular (las cuales se balancean de forma automática) y la ruta con el nombre de salida (por defecto, **`data/raw/data_vin_raw.parquet`**). Se ha elegido la extensión *.parquet* por cuestiones de almacenamiento y memoria.

Esta configuración es editable pero es recomendable mantenerla para un correcto flujo y conexión del bloque de encapsulamiento. Para llevarse a cabo la ejecución de estos archivos se puede realizar directamente desde los scripts o situándonos en la raíz del proyecto utilizar los comandos (ver Sección 9.2):

```
python scripts/data/descargar_clima_historico.py
python scripts/data/data_cleaner.py
python scripts/data/vid_simulator.py
```

3.6 Pipeline de ETL

Fase	Paso	Descripción
1	Descarga e Importación de datos	Obtención de los datos meteorológicos mediante <i>scripts/data/descarga_clima_historico.py</i> , limpieza con <i>scripts/data/data_cleaner.py</i> y lectura de los datos (.parquet) simulados mediante el archivo <i>scripts/data/vid_simulator.py</i> .
2	Asegurar Formato correcto de las columnas	Se ordenan alfabéticamente las columnas y se asegura que las variables de etiqueta sean de tipo <i>string</i> . De esta labor se encarga <i>src/data_processing/preprocess.py</i> .
3	Ingeniería de características	Se crean las variables derivadas: Hora, Hora_Sin, Hora_Cos, GDD_Acumulado y Horas_Hum_Foliar. De esta labor se encarga <i>src/data_processing/preprocess.py</i> .
4	Selección de variables	Escogemos las variables predictoras y descartamos aquellas que sean <i>targets</i> o <i>identificadores</i> . De esta labor se encarga <i>src/training/train.py</i> .
5	Codificación de <i>Targets</i>	Mediante un <i>Label Encoder</i> se transforman las etiquetas de <i>Clase_Entrenamiento</i> , haciendo que el <i>target</i> del modelo sea Etiqueta_Num . De esta labor se encarga <i>src/training/train.py</i> .
6	Particionado de Datos (<i>Split</i>)	Se divide el <i>dataset</i> de 989 series en 3 conjuntos disjuntos de series temporales completas por identificador de cada serie (ID_Serie) de forma aleatoria y estratificada. Es decir, se separa el conjunto de datos (utilizando la semilla 42 para reproducibilidad) en: - Entrenamiento: 70% Total: 692 series. - Validación: 15% Total: 148 series. - Test: 15% Total: 149 series. Además, se exporta a la carpeta <i>data/splits/</i> cada una de estas particiones. De esta labor se encarga <i>src/training/train.py</i> .

7	Escalado	Se ajusta y se guarda un escalador (StandardScaler) y un label encoder, utilizando solo el conjunto de entrenamiento y se aplica al <i>dataset</i> . De esta labor se encarga <i>src/training/train.py</i> .
8	Generación de ventanas temporales y tensores	Se crean ventanas deslizantes de un número editable de horas de contexto (editable desde <i>config/config.yaml</i>) y se empaquetan los datos en tensores tridimensionales para introducirlos a la red. De esta labor se encarga <i>src/training/train.py</i> .
9	Limpieza de RAM	Con <i>Garbage Collector</i> se libera memoria.

TABLA 2 PIPELINE DE ETL

4 Especificaciones Técnicas

4.1 Arquitectura y Algoritmos

El sistema propuesto se basa en una arquitectura de **Deep Ensemble Learning (DEL)** fundamentada en **3 Redes Neuronales con la misma ponderación**. Toda la lógica de arquitectura del modelo se encuentra en *src/training/train.py*, mientras que toda la base experimental se encuentra en el *notebook* con ruta *Modelos/Modelo_DEL.ipynb*. Tras diferentes pruebas comparativas (con *baselines* como *XGBoost* y redes profundas individuales), las arquitecturas escogidas para formar esta "fusión" de modelos fueron:

- **LSTM (Long Short-Term Memory):** Red neuronal recurrente (RNN) equipada con un sistema de "puertas" internas que le permite recordar u olvidar información a lo largo del tiempo. Es la opción estándar para datos secuenciales largos, ya que resuelve el problema de olvidar información lejana, siendo vital cuando el contexto histórico a largo plazo es clave para la predicción.
- **1D-CNN (Red Neuronal Convolutacional 1D):** Red que aplica filtros matemáticos unidimensionales a lo largo de una secuencia de datos para extraer características espaciales o temporales. Es computacionalmente muy rápida y excelente para identificar patrones locales, formas específicas o picos repentinos dentro de señales o series temporales de forma independiente a su posición.

- **BiGRU (Bidirectional Gated Recurrent Unit):** Red neuronal recurrente que procesa la secuencia de datos de forma simultánea en ambas direcciones (hacia adelante y hacia atrás) utilizando celdas GRU (una versión más ligera que las LSTM). Se utiliza cuando comprender el contexto futuro es tan importante como el pasado para evaluar el momento actual. Al ser GRU, ofrece un gran equilibrio entre velocidad de entrenamiento y alta retención de memoria.

La **estrategia de ejecución de las predicciones** es la siguiente:

1. Se recibe un contexto (1 semana = 168 horas) histórico de las variables predictoras.
2. Cada uno de los modelos lo analiza y genera 2 predicciones:
 - a. **Clasificación** del estado de la vid: **Etiqueta_Num**.
 - b. **Regresión** del grado de infección: **Grado_Infeccion**.
3. Se recopilan las predicciones individuales y se procesan para dar una predicción conjunta de la siguiente forma:
 - a. La predicción de la **regresión** será la **media aritmética** de las 3 predicciones.
 - b. Para **clasificación**, se emplea la estrategia de **Soft Voting** para decidir la etiqueta predicha, esta técnica considera la confianza de cada modelo en cada clase, realiza la media de cada una y selecciona la más alta.

De esta forma se reducen los errores que producen los modelos individualmente, obteniendo una mayor robustez y capacidad de generalización.

4.2 Entradas y Salidas

Entradas:

- **Tipo de Datos:** Series temporales multivariantes agroclimáticas de parcelas de vid.
- **Formato Físico:** DataFrame de Pandas en memoria, archivo Parquet estructurado con registros horarios procedentes de sensores IoT.
- **Estructura Esperada:**
 - **Series temporales** continuas de al menos **1 semana (168 horas)** de **registros con granularidad horaria** (1 registro por hora).

- **Variables crudas mínimas requeridas:** (9 + Identificador)
 - Temp_Amb_C
 - Hum_Rel_Pct
 - Lluvia_mm
 - Viento_kmh
 - Hum_Suelo_Pct
 - pH_Suelo
 - CO2_ppm
 - VOC_ppb
 - Fecha (*timestamp*).
- **Identificador (Opcional, pero recomendado):** En caso de contener una columna "ID_Serie" (clave única de cada serie) se utilizará para agrupar las diferentes series y realizar las predicciones sobre cada una de ellas.
- **NOTA: Si no se dispone de alguna de las variables mínimas se debe eliminar del archivo de configuración (config/config.yaml).** Estas variables son las mínimas recomendadas y evaluadas durante este proyecto, en caso de eliminar alguna de ellas las métricas de rendimiento podrían decaer (como se observa en la Tabla 9).

Salidas:

- **Tipo de Salida:** El sistema genera dos salidas:
 - **Clasificación multiclase:** Predicción *Soft Voting* de la patología más probable detectada entre 11 posibles (Black Rot, Botrytis, Oídio, Mildiu, Esca, Lobesia Botrana, Empoasca, Erinosis, Altica, Araña Roja o Planta Sana).
 - **Regresión continua:** Valor medio que indica el grado de infección de dicha patología.
- **Formato:** Archivo CSV (*data/predictions/inferencia_vid.csv*) con las siguientes columnas:
 - **Fecha_Evaluacion:** Marca temporal correspondiente al final de la ventana analizada (hora actual de la predicción).
 - **Diagnostico_IA:** Etiqueta en texto con el nombre de la plaga o enfermedad diagnosticada.
 - **Confianza_Clasificacion:** Probabilidad porcentual de la clase ganadora predicha por el *Deep Ensemble*.

- **Grado_Severidad:** Valor continuo acotado entre 0 y 1 que modela el nivel de daño detectado.
- **Tratamiento_Recomendado:** Texto que incluye el protocolo de actuación químico, biológico, biotecnológico y cultural pertinente según la base de conocimiento (basada en [2] pero editable desde *config.yaml*).
- **ID_Serie:** En caso de hacer la predicción sobre todo el conjunto de datos conteniendo identificadores únicos, se acompaña cada predicción con su identificación correspondiente.

4.3 Requisitos de Hardware y Entorno

El desarrollo, entrenamiento y evaluación del modelo DEL se ha realizado en un entorno de *Python* 3.10.11 ejecutado sobre *Windows 11 Pro*. El uso de un entorno virtual junto con un fichero *requirements.txt* permite replicar esta configuración en otras máquinas, garantizando la reproducibilidad del ciclo completo de entrenamiento e inferencia. **En caso de que el sistema no detecte una GPU compatible con CUDA, el entrenamiento pasará automáticamente a ejecutarse en la CPU** de forma transparente (fallback), aunque esto supondrá un incremento en el tiempo de procesamiento.

Hardware de Referencia y Mínimos Recomendados:

- **CPU:** Procesador Intel Core i9-14900K (3.20 GHz) utilizado en el desarrollo. Para replicar el entorno, se recomienda un procesador estándar x86-64 de alto rendimiento (mínimo 4-8 núcleos), necesario para el preprocesamiento intensivo y la ingeniería de características por ciclo.
- **Memoria RAM:** 64 GB instalados en el equipo de referencia. Se establece un mínimo estricto de 50 GB para replicar el proyecto, garantizando capacidad suficiente para cargar el histórico de sensores y gestionar los tensores tridimensionales.
- **GPU:** Altamente recomendada para la fase de entrenamiento. El modelo emplea una arquitectura DEL que combina 3 modelos de aprendizaje profundo. Se recomienda una GPU compatible con NVIDIA CUDA. La inferencia unitaria, sin embargo, puede ejecutarse de forma eficiente en CPU.
- **Almacenamiento:** Mínimo de 5 GB para alojar el código fuente, los datasets crudos y procesados, y los artefactos exportados (ficheros csv, parquet, pkl o keras).

En la Tabla 3 se documentan, a modo de referencia, los tiempos medios de entrenamiento e inferencia obtenidos con el hardware empleado durante todo el proceso de desarrollo.

Proceso	Duración	Hardware (CPU • GPU • RAM)
Entrenamiento (pipeline tabular completo)	80 minutos (aproximadamente)	Intel(R) Core(TM) i9-14900K (3.20 GHz) • NVIDIA GeForce RTX 4070 • 64 GB RAM
Evaluación	25 segundos	
Inferencia	0.2 segundos	

TABLA 3. TIEMPOS DE ENTRENAMIENTO E INFERENCIA CON HARDWARE DE REFERENCIA

Librerías y herramientas principales:

- Librerías núcleo (producción):
 - **tensorflow / keras:** *Framework* principal para el entrenamiento, construcción y validación del *Deep Ensemble Learning*. Provee la arquitectura de redes neuronales convolucionales (1D-CNN) y convoluciones recurrentes (LSTM, GRU Bidireccional) para el análisis secuencial.
 - **pandas:** Gestión integral y transformación de la base de datos tabular, lectura veloz de formatos .parquet y .csv, y pivoteo de series temporales.
 - **numpy:** Operaciones matriciales subyacentes de alta eficiencia, como la generación de características cíclico-temporales (Coseno/Seno) y operaciones de probabilidad y *Soft Voting* por encima de los tensores tridimensionales.
 - **scikit-learn:** Lógica fundacional del *pipeline* con herramientas para particionado estratificado (`train_test_split`), escalado de características (`StandardScaler`), y codificación discreta de la variable objetivo o patología (`LabelEncoder`).
 - **joblib:** Serialización binaria rápida y volcado de modelos preprocesadores (*Scalers* y *Encoders*) durante el ciclo de vida del *deployment*.
 - **pathlib:** Tratamiento agnóstico y robusto del sistema de ficheros para rutas dinámicas entre la carga en crudo y el guardado de artefactos del modelo.

- **pyarrow / fastparquet:** Motores de descompresión columnares necesarios para ingerir y salvaguardar los grandes fragmentos particionados del *Dataset* en discos duros.
- **requests:** Para extracción de variables meteorológicas de la API.
- Librerías para experimentación y análisis exploratorio:
 - **matplotlib / seaborn:** Representación cruzada de distribuciones temporales, generación de las curvas de aprendizaje, y mapeado térmico (*Heat Map*) de las matrices de confusión diagnóstica.
 - **tqdm:** Análisis de traza y representación de las barras de progreso métricas durante las operaciones asíncronas de empaado de tensores 3D.
 - **gc:** Recolección implícita de basura para salvaguardar fugas críticas de Memoria RAM durante el modelado de ventanas rodantes masivas.

5 Análisis y Selección de Modelos

La evaluación del rendimiento de las arquitecturas propuestas se ha estructurado en torno a dos tareas dentro del paradigma de aprendizaje multitarea: la clasificación taxonómica de la patología y la regresión del grado de severidad continua de la infección. Con el propósito de evaluar la viabilidad del sistema experto y justificar la complejidad computacional adoptada, se contrastó el rendimiento de un algoritmo basado en árboles de decisión ensablados (XGBoost) y tres arquitecturas de redes neuronales profundas base (LSTM, 1D-CNN y BiGRU) frente a la arquitectura consolidada de Aprendizaje en Ensamble Profundo (DEL).

5.1 Resultados Comparativos Clasificación

El comportamiento de los modelos ante el problema de clasificación multiclase (11 estados fitosanitarios del conjunto de test hold-out) se detalla en la Tabla 4.

Modelo	Accuracy	Precisión (Macro)	Recall (Macro)	F1-Score (Macro)
XGBoost	0,4417	0,4626	0,4513	0,4441
LSTM	0,6748	0,7318	0,6993	0,7063
1D-CNN	0,6874	0,7199	0,6965	0,7033

BiGRU	0,6444	0,6504	0,6384	0,6382
DEL	0,7052	0,7494	0,7198	0,7259

TABLA 4 COMPARATIVA DE MÉTRICAS EN EL CONJUNTO DE TEST PARA CLASIFICACIÓN

Como se demuestra en la Tabla 4, el algoritmo básico tabular XGBoost exhibe un rendimiento deficiente, registrando un F1-Score Macro de apenas 0,4441. Esta degradación evidencia la incapacidad estructural de los modelos basados en árboles estáticos para ingerir tensores crudos secuenciales de 168 horas sin un aplanamiento (flattening) masivo que destruya la topología y el orden cronológico continuo del microclima. Por el contrario, las arquitecturas de Deep Learning individuales manifiestan una notable superioridad. La red 1D-CNN y la puerta recurrente LSTM operan como los mejores estimadores independientes, logrando F1-Scores de 0,7033 y 0,7063 respectivamente. No obstante, la arquitectura unificada DEL se erige como la opción óptima en todas las métricas analizadas en la Tabla 4, alcanzando una Exactitud Global (Accuracy) de 0,7052 y un F1-Score Macro de 0,7259. Este resultado valida empíricamente el cumplimiento del criterio técnico de éxito exigido para la clasificación ($\geq 0,70$), demostrando que la combinación de múltiples sesgos inductivos mediante Soft Voting neutraliza las clasificaciones erróneas marginales de las redes base.

5.2 Resultados Comparativos Regresión

El análisis del desempeño continuo para la estimación del target analógico (Grado_Infeccion), evaluado sobre las mismas particiones de test independientes, se expone en la Tabla 5.

Modelo	MAE	RMSE	R2
XGBoost	0,0931	0,1273	0,7969
LSTM	0,0700	0,1066	0,8575
1D-CNN	0,0779	0,1149	0,8346
BiGRU	0,0786	0,1170	0,8283
DEL	0,0610	0,0936	0,8903

TABLA 5 COMPARATIVA DE MÉTRICAS EN EL CONJUNTO DE TEST PARA REGRESIÓN

Los resultados de la Tabla 5 demuestran que, si bien todas las redes neuronales superan holgadamente la varianza explicada por el baseline tabular (XGBoost), existe disparidad en su capacidad analítica individual para modelar la severidad. La

arquitectura BiGRU y la 1D-CNN exhiben un rendimiento sólido pero discreto (R^2 de 0,8283 y 0,8346 respectivamente), quedando ligeramente por debajo del umbral agronómico objetivo. Por su parte, la red LSTM sí logra aislar correctamente el patrón de incubación a largo plazo, alcanzando un R^2 individual de 0,8575. El ensamble DEL capitaliza los aciertos individuales y corrige sus desviaciones de forma sobresaliente. De acuerdo con los datos de la Tabla 5, la agregación matricial de las predicciones mitiga drásticamente el error promedio, reduciendo el Error Medio Absoluto (MAE) a 0,0610 y comprimiendo la varianza residual (RMSE) a 0,0936. En términos de precisión global, el modelo DEL alcanza un Coeficiente de Determinación (R^2) de 0,8903, superando con amplia solvencia el umbral mínimo de validación fijado en los objetivos del proyecto ($R^2 \geq 0,85$).

5.3 Conclusiones de la Comparativa

Los resultados empíricos expuestos en las tablas justifican de manera concluyente la selección e implementación productiva de la arquitectura Deep Ensemble Learning (DEL) apoyándose en tres pilares técnicos:

- **Sinergia de Extracción de Características:** El éxito del modelo DEL radica en su capacidad para explotar de forma simultánea las virtudes topológicas de sus componentes. Mientras que la 1D-CNN extrae eficientemente alteraciones microclimáticas instantáneas, la LSTM procesa la inercia bioclimática ininterrumpida. Esta sinergia matemática permite al ensamble capturar de forma holística interacciones complejas que escapan a las redes individuales.
- **Robustez y Mitigación de la Varianza (Regularización por Ensamble):** Aunque redes como la BiGRU o la 1D-CNN mostraron F1-Scores competitivos (Tabla 4), presentaron ciertas limitaciones a la hora de estabilizar la regresión (Tabla 5). Al integrar los tres flujos bajo un esquema de promediado y Soft Voting, el ensamble DEL actúa como un poderoso regularizador natural, amortiguando los errores puntuales de una red aislada y elevando el techo de rendimiento más allá del límite del mejor modelo individual.
- **Cumplimiento Integral de los Criterios de Éxito:** El modelo DEL se consolida como la única arquitectura evaluada capaz de satisfacer simultáneamente y con holgura las dos exigencias matemáticas del sistema: un F1-Score Macro superior al objetivo (0,7259) y un ajuste de la varianza casi excelente en regresión ($R^2 = 0,8903$).

Se concluye, por tanto, que la arquitectura ensamblada proporciona el equilibrio algebraico idóneo entre precisión diagnóstica multiclase y estabilidad predictiva

continua. Esto la consolida como la metodología óptima para dar soporte fiable a las decisiones de campo y garantizar el cumplimiento del KPI de negocio (reducción de la huella fitosanitaria).

6 Proceso de Entrenamiento y Evaluación

6.1 Descripción del los Procesos

Una vez consolidado el conjunto de datos a través del pipeline de ETL, se procede al diseño conceptual y computacional del bucle de entrenamiento del ensamble de aprendizaje profundo (Deep Ensemble Learning, DEL). El modelado se afronta bajo el paradigma de aprendizaje multitarea (Multi-task Learning), donde el espacio latente de características procesado por los modelos se bifurca de manera coordinada para optimizar una función de pérdida compuesta. Cada observación dentro del tensor tridimensional de entrada representa una ventana temporal deslizante de 1 semana (168 horas), mapeada hacia una etiqueta dual que define concurrentemente:

- El tipo específico de patología/plaga detectada (**clasificación multiclase**).
- La severidad de dicha infección, de 0 a 1 (**regresión continua**).

Sobre esta estructura tensorial se ajustan de manera secuencial e independiente tres arquitecturas neuronales profundas con sesgos inductivos (inductive biases) complementarios: una red de memoria a largo plazo (LSTM), experta en aislar inercias bioclimáticas continuas; una red convolucional unidimensional (1D-CNN), optimizada para la detección de gradientes o picos impulsivos locales; y una red recurrente profunda bidireccional (BiGRU), encargada de extraer dependencias contextuales rompiendo la causalidad temporal estricta.

Las tres redes ingieren el bloque idéntico de 12 características predictoras numéricas. Para mitigar de raíz el sesgo de selección y la fuga de información (data leakage), los parámetros de centralización y escala del preprocesador (StandardScaler) se calculan **única y exclusivamente** sobre la matriz del conjunto de entrenamiento, aplicándose de forma pasiva sobre las particiones de validación y test hold-out. El mecanismo de agregación final unifica los vectores de salida mediante una ponderación uniforme ($w_m = \frac{1}{3}$). Las funciones de probabilidad Softmax de clasificación se consolidan aplicando un filtrado de votación suave (Soft

Voting), mientras que las salidas predictoras continuas de la función Sigmoide de regresión se unifican mediante la media aritmética simple de los estimadores.

6.2 Hiperparámetros y Arquitectura Clave

El sistema opera bajo un conjunto de parámetros estructurales y dinámicos, consolidados tras el proceso de experimentación, validación cruzada y *Fine-Tuning*.

Pruebas para selección de técnicas sensibles y características del modelo

Para seleccionar diferentes aspectos clave del modelo como el tamaño de la ventana de contexto, tipo de escalador y tipo de padding para inferencia, se realizaron una serie de experimentos detallados en el notebook [notebooks/Modelos/Pruebas_Modelo_DEL.ipynb](#). Cada configuración se evaluó midiendo su impacto directo sobre las métricas del conjunto de Validación, garantizando que el diseño final responda a evidencias empíricas y no a parámetros por defecto, sin utilizar en ningún momento el conjunto de Test para no sobrevalorar al modelado. Todo esto se realizó antes de la optimización de hiperparámetros.

Tamaño de Ventana (Contexto temporal)

Para determinar el marco histórico óptimo que el modelo necesita para capturar la incubación de patologías, se evaluó el rendimiento del ensamble ingiriendo secuencias temporales de **7, 10 y 14 días (168, 240 y 336 horas** respectivamente). Los datos de la Tabla 6 demuestran que el horizonte de 1 semana (168 horas) ofrece el mejor equilibrio algebraico y operativo. Extender el contexto temporal a 2 semanas incrementa el coste computacional en un 123.9% (de 46 a 103 minutos) y degrada el F1-Score Macro a 0,7193 debido a la introducción de ruido histórico de largo alcance que diluye los gradientes instantáneos de infección.

Window Size	Accuracy	Precision Macro	Recall Macro	F1-Score Macro	R2	Tiempo Total de Train
1 semana	0,7475	0,7709	0,7306	0,7428	0,8749	46 min
10 días	0,7455	0,7713	0,7238	0,7360	0,8747	75 min
2 semanas	0,7598	0,7879	0,7008	0,7193	0,8611	103 min

TABLA 6 MÉTRICAS DEL DEL EN VALIDACIÓN PARA VARIOS VALORES DEL TAMAÑO DE LA VENTANA DE CONTEXTO

Escalador (Distribución de variables)

Debido a la disparidad de magnitudes físicas en los sensores (por ejemplo, concentraciones de VOCs frente a porcentajes de humedad), se analizó el impacto de diferentes técnicas de normalización de datos. Se comparó el rendimiento del modelo utilizando un **StandardScaler** frente a un **MinMaxScaler**.

De acuerdo con la Tabla 7, la implementación de StandardScaler (z-score) fue seleccionada formalmente al inducir un incremento significativo en el F1-Score Macro (0,7428 frente a 0,7195). Centrar las características en media cero y varianza unitaria aporta una mayor estabilidad matemática a los gradientes de retropropagación frente a valores climáticos extremos.

Tipo de Scaler	Accuracy	Precision Macro	Recall Macro	F1-Score Macro	R2
StandardScaler	0,7475	0,7709	0,7306	0,7428	0,8749
MinMaxScaler	0,7307	0,7531	0,7157	0,7195	0,8762

TABLA 7 MÉTRICAS DEL DEL EN VALIDACIÓN PARA VARIOS ESCALADORES

Estrategia de Padding (Inferencia)

Dado que el sistema en producción requiere un tensor estricto de 168 horas, se simuló un escenario de pérdida de telemetría (con falta de 1, 2 y 3 días de datos) para evaluar cómo la imputación algorítmica de los datos faltantes degrada el modelo. Se contrastaron tres aproximaciones: *Edge Padding* (clonar el primer valor), *Zero Padding* (inyectar ceros/anomalías) y *Mean Padding* (inyectar la media histórica). Como se observa en la Tabla 8, se adoptó el *Padding por Repetición (Edge Padding)* de forma definitiva. Aunque la falta de datos degrada la precisión ideal, esta estrategia amortigua drásticamente la caída del F1-Score, ya que mantiene la continuidad física de la serie termodinámica y evita introducir "escalones" matemáticos artificiales que confundan a las redes convolucionales.

Días Faltantes	Tipo de Padding	F1-Score Macro	RMSE
0 días	Datos reales (Baseline)	0,7428	0,0955

1 día	Padding Por Repetición	0,3803	0,1974
	Zero Padding	0,0713	0,2926
	Mean Padding	0,1763	0,3254
2 días	Padding Por Repetición	0,3002	0,2330
	Zero Padding	0,0317	0,3481
	Mean Padding	0,0634	0,3513

TABLA 8 MÉTRICAS DEL DEL EN VALIDACIÓN PARA VARIAS TÉCNICAS DE PADDING

Inclusión/Exclusión de variable de VOC en cuanto a rendimiento

Para cuantificar el valor predictivo de las emisiones de compuestos orgánicos volátiles, se llevó a cabo un estudio excluyendo la variable VOC_ppb del conjunto de entrenamiento (ver notebook [Modelo_DEL_sin_VOC.ipynb](#)). Los resultados empíricos, recogidos en la Tabla 9, evidencian una penalización notable en la capacidad de detección temprana del algoritmo. Esta degradación de las métricas confirma que la variable captura información metabólica crítica sobre el estrés biótico de la planta, justificando plenamente la inversión en este tipo de sensores físicos para garantizar un rendimiento óptimo en un entorno de producción.

Variables Usadas	Accuracy	F1-Score (Macro)	Precision (Macro)	Recall (Macro)	R2
Todas	0,7052	0,7259	0,7494	0,7198	0,8903
Sin VOC_ppb	0,5849	0,5396	0,5872	0,5452	0,8498

TABLA 9 MÉTRICAS DEL DEL EN TEST CON Y SIN LA VARIABLE VOC

No obstante, el sistema está diseñado bajo un enfoque modular para adaptarse a diferentes niveles de infraestructura agrícola. En escenarios donde la instalación de sensórica de gases en campo no sea viable por restricciones logísticas o presupuestarias, el cliente tiene la opción de operar bajo el escenario restrictivo expuesto en la Tabla 9. Para efectuar este despliegue, la transición es directa a nivel de código: bastaría con omitir VOC_ppb (u otras variables de las que no se disponga en producción) de la lista de predictores en el archivo de configuración

(**config/config.yaml**) y ejecutar un nuevo entrenamiento del modelo adaptado a la sensorica disponible.

Configuración global

- **window_size = 168:** Define el contexto de la memoria del modelo. Corresponde a 168 horas (7 días) ininterrumpidas de registro histórico, conformando un tensor de entrada tridimensional (Muestras x 168 horas x 12 variables físicas).
- **random_seed = 42:** Semilla global fijada para garantizar la reproducibilidad estricta en la partición de datos y la inicialización de los pesos neuronales.
- **BATCH_SIZE = 512:** Tamaño de lote, para una gestión eficiente de los tensores en memoria.

Hiperparámetros Óptimos por Arquitectura

Cada red del ensamblaje (*Ensemble*) posee una configuración única adaptada a su naturaleza de extracción de características. Estas arquitecturas se optimizaron mediante el notebook **Optimizacion_DEL_CrossValidation.ipynb**. Este proceso se llevó a cabo tras la selección del tamaño de la ventana de contexto, el escalador y el padding. Las arquitecturas de cada red son las siguientes:

- **LSTM** (Especialista en memoria a largo plazo)
 - **Estructura:** Dos capas recurrentes de 8 y 32 unidades.
 - **Tasa de aprendizaje:** 0,001
 - **Regularización (Dropout):** 0,2. Posee la tasa de "olvido" ajustada para evitar la memorización estricta del histórico agroclimático sin perder información valiosa.
- **1D-CNN** (Especialista en detección de picos locales)
 - **Estructura:** Dos capas convolucionales de 16 y 32 filtros, con un tamaño de ventana de visión (*Kernel*) de 3x3.
 - **Tasa de aprendizaje:** 0,001
 - **Regularización (Dropout):** 0,2.
- **BiGRU** (Especialista en contexto temporal bidireccional)
 - **Estructura:** Capa recurrente bidireccional de 16 unidades, conectada a una capa densa de 8 neuronas.

- **Tasa de aprendizaje:** 0,001.
- **Regularización (Dropout):** 0,2.

Optimizador y función de pérdida

- **Optimizador:** Adam, elegido por su eficiencia mitigando gradientes explosivos o desvanecientes en redes profundas.
- **Vigilancia (Early Stopping):** Paciencia de 2 épocas sobre la pérdida de validación para abortar el entrenamiento en el punto algebraico óptimo, cortando de raíz cualquier indicio de *overfitting*.
- **Función de Pérdida Dual:** Las redes resuelven dos problemas de forma simultánea e independiente, ponderados de forma equilibrada (peso 1.0):
 - Para la salida de clase: **sparse_categorical_crossentropy**.
 - Para el grado de infección: **mse** (Error Cuadrático Medio).

Método de optimización de hiperparámetros

Los hiperparámetros escogidos para el modelado se determinaron mediante una estrategia de dos fases diseñada para equilibrar velocidad computacional y rigor estadístico. Toda la lógica de optimización se encuentra en [*tuning_hyperparameters/Optimizacion_DEL_CrossValidation.ipynb*](#).

- **Fase 1 (Exploración mediante Random Search):** El algoritmo evaluó 10 combinaciones (N_TRIALS) para cada arquitectura utilizando una única partición de control (80% entrenamiento, 20% validación), aplicando un mecanismo de Early Stopping=2 para abortar y descartar rápidamente las configuraciones ineficientes.
- **Fase 2 (Validación Cruzada):** Una vez identificada la mejor combinación estructural y los hiperparámetros de cada red, cada modelo se sometió a una validación cruzada de 5 *Folds* sobre la totalidad de los datos (sin el conjunto de Test). Este paso final fue clave para certificar que la precisión obtenida era sólida y no fruto de un reparto afortunado de los datos, garantizando la capacidad de generalización del modelo sin mezclar jamás la serie temporal de una misma planta.

6.3 Estrategia de Validación y Splits

La validación del modelo se cimienta de forma estricta sobre una estrategia de separación espacial estratificada por grupos independientes (Holdout por Grupo).

Debido a la naturaleza biológica de las observaciones, donde los registros horarios pertenecen a series temporales que describen el ciclo evolutivo completo de una cepa específica, un particionado puramente aleatorio por filas provocaría un desastre metodológico al distribuir registros horarios de una misma planta entre entrenamiento y prueba.

Para anular por completo el riesgo de fuga de datos temporal, el particionado bloquea las series completas utilizando el identificador unívoco ID_Serie bajo una semilla constante (random_seed 42). El dataset neto de 989 series se segmenta en tres particiones disjuntas:

- **Conjunto de Entrenamiento (70% | 692 series completas):** Destinado al ajuste de la matriz de pesos y bias de las tres redes del ensamble.
- **Conjunto de Validación (15% | 148 series completas):** Utilizado de forma pasiva para la monitorización de las curvas de aprendizaje y la activación de los callbacks de parada.
- **Conjunto de Test Hold-Out (15% | 149 series completas):** Conjunto ciego aislado en un contenedor externo, utilizado exclusivamente para el reporte final de métricas operativas.

Esta estratificación garantiza que el sistema experto evalúe la capacidad del Deep Learning para abstraer las leyes bioclimáticas generales de la infección (interacción de Grados-Día y Humedad) y no la simple memorización de trayectorias históricas ya observadas.

Los **splits** se generan automáticamente al ejecutar el entrenamiento y se realizan con una semilla fija (42) para poder reproducir el flujo completo. Por cuestiones de almacenamiento, la carpeta de datos (splits, datos en crudo, procesados...) **no se ha podido incluir en el repositorio por lo que se encuentran en un contenedor externo.**

6.4 Orquestación y Criterios de Parada

El ciclo completo (preprocesamiento dimensional, particionado de tensores, entrenamiento secuencial, evaluación cruzada e inferencia en batch) se orquesta centralizadamente desde el CLI principal o módulos encapsulados como *scripts/train.py*. Desde el punto de vista operativo, toda transformación del conjunto y serialización física a archivos .keras y meta-herramientas .pkl están totalmente automatizadas tras invocar un simple sub-comando.

Los criterios de parada actúan por cada red de forma independiente:

- **EPOCHS = 50:** Límite superior rígido de iteraciones operativas sobre la totalidad del conjunto por cada sub-modelo.
- **Early Stopping (Patience) = 2:** Si durante dos bucles consecutivos el valor de la métrica de Validación Global (val_loss) no decae logarítmicamente, el entrenamiento de la red activa aborta.
- **Restore Best Weights / ModelCheckpoint:** Al desencadenarse un criterio de estancamiento, el sistema no entrega en producción los pesos finales sobreajustados de la última época, sino que retrocede en la memoria gráfica local y restaura automáticamente el exacto *sub-checkpoint* donde el rendimiento de Generalización fue empíricamente óptimo.

7 Métricas y Rendimiento

7.1 Criterios y Métricas de Éxito

El modelo dual (multiclase y regresión) se evalúa mediante métricas estándar conjuntas para series temporales univariantes y multivariantes. Como ya se comentó en la Sección 2, se establecen los siguientes criterios cuantitativos de éxito sobre el diagnóstico de la patología y su grado de infección:

- Clasificación: **F1-Score (Macro) $\geq 0,70$**
- Regresión: **R2 $\geq 0,85$**

Métricas y criterios de éxito para Clasificación

La elección de estos umbrales se apoya en el caso de uso y el impacto económico directo sobre el viñedo. Al no disponer de registros reales acerca del uso de productos o gastos en viñedos, el objetivo establecido será maximizar las métricas:

- **Recall (Macro):** En el contexto de plagas muy agresivas (como el Black Rot o el Mildiu), el coste de un falso negativo (no detectar la infección a tiempo) significa la pérdida potencial de toda la cosecha de la parcela. Su expresión matemática es la siguiente:

$$Recall_i = \frac{TP_i}{TP_i + FN_i}$$

$$Recall Macro = \frac{1}{N} \sum_{i=1}^N Recall_i$$

donde:

- TP_i : Verdaderos positivos para la clase i .

- FN_i : Falsos Negativos para la clase i .
 - N : Número total de clases (en nuestro caso $N = 11$ estados).
- **Accuracy:** Aunque es una métrica engañosa de forma aislada, cruzada con los dos criterios anteriores asegura que, en volumen neto total de predicciones horarias, el sistema no somete a fatiga o estrés operativo al agricultor al fallar frecuentemente sobre el diagnóstico base. Su formulación matemática es la siguiente:

$$Accuracy = \frac{\sum_{i=1}^N TP_i}{Total\ de\ Muestras}$$

donde:

- TP_i : Verdaderos positivos para la clase i .
 - N : Número total de clases (en nuestro caso $N = 11$ estados).
- **F1-Score (Macro):** En sanidad vegetal, el dataset está inherentemente desbalanceado (las epidemias, como el Oídio o la Lobesia, son eventos anómalos frente al estado fisiológico "Sano"). Una exactitud (Accuracy) elevada podría lograrse simplemente prediciendo siempre "Sano", ocultando fallos catastróficos. El F1-Score Macro otorga el mismo peso a todas las clases independientemente de su soporte estadístico. Un umbral $\geq 0,80$ garantiza un equilibrio operativo sólido: una alta fiabilidad en las alertas emitidas (Precision) combinada a una alta tasa de captura (Recall), minimizando drásticamente la aplicación errónea de fungicidas caros y tóxicos por falsas alarmas. La expresión matemática es la siguiente:

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

$$F1_i = 2 \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i}$$

$$F1\ Macro = \frac{1}{N} \sum_{i=1}^N F1_i$$

donde:

- TP_i : Verdaderos positivos para la clase i .
- FP_i : Falsos Positivos para la clase i .

- N : Número total de clases (en nuestro caso $N = 11$ estados).

Los tres umbrales garantizan simultáneamente rentabilidad operativa, sostenibilidad química (reduciendo tratamientos innecesarios) y máxima cobertura fitosanitaria.

Métricas y criterios de éxito para Regresión

Para evaluar el desempeño del modelo en la predicción continua del grado de infección de la planta, se implementó un panel de métricas de regresión complementarias que garantizan la fiabilidad cuantitativa del sistema. El **Error Medio Absoluto (MAE)** se utilizó para medir la magnitud promedio de la desviación en las predicciones, ofreciendo una interpretación directa del error esperado en la escala de severidad. Para asegurar que el modelo no cometiera fallos graves por exceso o defecto, se calcularon el **Error Cuadrático Medio (MSE)** y su raíz (**RMSE**), métricas matemáticamente diseñadas para penalizar con mayor dureza las discrepancias grandes. Finalmente, el **Coefficiente de Determinación (R^2)** permitió cuantificar qué porcentaje de la varianza en la progresión de la enfermedad es correctamente explicado por las variables agroclimáticas introducidas, validando así la robustez y capacidad analítica del modelo a la hora de predecir el daño real en el cultivo. Formalmente, tendremos lo siguiente:

Sea N el número total de muestras, y_i el valor real del grado de infección de la planta i , $\hat{y}_i$ el valor predicho e $\bar{y}$ la media aritmética de todos los valores reales, tendremos:

- Error Absoluto Medio: $MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$
- Error Cuadrático Medio: $MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
- Raíz del Error Cuadrático Medio: $RMSE = \sqrt{MSE}$
- Coeficiente de determinación: $R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$

7.2 Validación cruzada y fine-tuning

Para garantizar la robustez y capacidad de generalización del sistema Deep Ensemble Learning (DEL), se ejecutó un proceso exhaustivo de validación cruzada agrupada acoplado a la **optimización de hiperparámetros (Fine-Tuning) mediante búsqueda aleatoria (Random Search)**. Las tres arquitecturas neuronales base (LSTM, 1D-CNN y BiGRU) fueron sometidas a este escrutinio

técnico para consolidar su configuración estructural con un total de **10 épocas en cada uno de los 10 intentos** (N_TRIALS) para cada arquitectura.

La **validación cruzada** se diseñó aislando estrictamente las series temporales por identificador utilizando el método de los **k-folds con k=5**. Esta técnica de agrupamiento es fundamental en el modelado de datos agronómicos secuenciales, ya que evita la fuga de información entre los conjuntos de entrenamiento y validación. A través de la Tabla 10, se observa que su aplicación evidencia que el alto rendimiento predictivo alcanzado por las redes no es producto de la memorización o el sobreajuste (overfitting), lo que confirma la estabilidad del modelo.

Categoría	Parámetro / Métrica	Valor
Mejores Hiperparámetros Encontrados	Learning rate	0,001 (Consistente en los 3 modelos)
	Dropout	LSTM: 0,2 CNN: 0,2 BiGRU: 0,2
	Capas Principales	LSTM: 8 y 32 unidades CNN: 16 y 32 filtros (Kernel 3x3) BiGRU: 16 unidades GRU y Densa 8
	Batch Size	512 (Consistente en los 3 modelos)
	Accuracy	LSTM: 0,7205 ($\pm 0,0184$) CNN: 0,7288 ($\pm 0,0118$) BiGRU: 0,6994 ($\pm 0,0202$)
Resultados de los K-Folds (K=5) con estos hiperparámetros	Loss Media	LSTM: 0,8433 CNN: 0,7399 BiGRU: 0,8387
Eficiencia de Búsqueda del Tuneado mediante Random Search	Entrenamientos evaluados	30 entrenamientos de tune (10 por modelo con 10 EPOCHS)
	Duración total	24 horas (aproximadamente)

TABLA 10. RESULTADOS DE VALIDACIÓN CRUZADA Y FINE TUNING DEL MODELO DEL

En conjunto, esta metodología de validación rigurosa demuestra que el modelo genera un entendimiento profundo del caso de uso. Los resultados en validación cruzada indican que la varianza de error entre utilizar un *fold* u otro es mínima, y por lo tanto, aseguran que **no se produzca sobreajuste** y que el rendimiento del modelo es óptimo independientemente del conjunto de entrenamiento empleado. Esto certifica su viabilidad operativa y fiabilidad estadística para el despliegue en producción, sin requerir mecanismos de supervisión manual adicionales.

7.3 Resultados en Conjunto de Test (Hold-Out)

Tras la configuración de la arquitectura y la validación cruzada, el modelo Deep Ensemble Learning (DEL) se evaluó sobre el conjunto ciego de Test (15% de las series, aisladas por ID_Serie). Los resultados cuantitativos globales para ambas tareas simultáneas se exponen en las Tablas 11 y 12.

Métrica Clasificación	Valor	Interpretación
Accuracy	0,7052	Exactitud global. El modelo acierta la patología correcta o el estado sano en el 70,52 % del total de los casos evaluados.
Precision (Macro)	0,7494	Fiabilidad de las alertas. Cuando el modelo predice que existe una enfermedad concreta, acierta el 74,94% de las veces. Esto minimiza los falsos positivos y evita gastar dinero en tratamientos innecesarios.
Recall (Macro)	0,7198	Capacidad de detección. El modelo identifica exitosamente el 71,98% de los casos reales de enfermedad. Es una tasa alta que asegura que muy pocas plantas enfermas pasen desapercibidas (pocos falsos negativos).
F1-Score (Macro)	0,7259	Equilibrio general. Muestra una relación excelente (72,59%) entre la precisión y la detección. Al ser "Macro", garantiza que el modelo predice bien todas las enfermedades por igual, sin ignorar las patologías menos frecuentes.

TABLA 11. MÉTRICAS DE CLASIFICACIÓN EN EL CONJUNTO DE TEST DEL MODELO DEL

Métrica Regresión	Valor	Interpretación
MAE	0,0610	Error promedio. Al predecir el porcentaje de daño, el modelo se desvía de media solo 0,0610 puntos (un 6,09% de error) respecto a la realidad. Es un margen de error mínimo y excelente para uso agronómico.
RMSE	0,0936	Desviación típica del error. Indica que el margen de error estándar se mueve en torno a un 9,36% en la misma escala de medida (0 a 1). Confirma que las estimaciones de severidad son altamente estables.
R2	0,8903	Explicación de la varianza. El modelo es capaz de entender y predecir el 89,03% del comportamiento real en la evolución de las infecciones. Representa un ajuste casi perfecto a los datos biofísicos.

TABLA 12. MÉTRICAS DE REGRESIÓN EN EL CONJUNTO DE TEST DEL MODELO DEL

Como se evidencia en las Tablas 11 y 12, el modelo cumple de forma íntegra los objetivos cuantitativos establecidos en el criterio de éxito del proyecto. El F1-Score Macro de 0,7259 supera el umbral mínimo exigido ($\geq 0,70$), ratificando la viabilidad de la arquitectura híbrida frente a la incertidumbre del ruido ambiental. Desde la perspectiva de la regresión, un coeficiente R2 de 0,8903 rebasa el umbral agronómico ($R2 \geq 0,85$), garantizando un soporte fiable para estimar las mermas de producción.

El uso del particionado estricto por ID_Serie demuestra concluyentemente que el modelo no ha memorizado coordenadas espurias, sino que ha abstraído las leyes bioclimáticas termodinámicas (temperatura, horas de humedad foliar, GDD) que rigen la fitopatología de la vid.

Análisis diagnóstico Multiclase

El escrutinio detallado de las predicciones revela un comportamiento dicotómico en el aprendizaje del modelo, fuertemente condicionado por la naturaleza etiológica de cada clase.

Patología Real	ALTICA	2577	0	0	38	0	0	126	0	0	118	0
	BLACK_ROT	0	19971	11357	4	900	32	2121	1399	3348	0	0
	BOTRYTIS	0	5371	16262	0	831	201	1068	1398	4988	0	0
	EMPOASCA	323	15	0	22517	40	0	32	41	0	197	0
	ERINOSIS	0	653	14	42	7381	122	933	2284	111	16	0
	ESCA	0	1	48	0	171	21515	76	242	0	0	0
	HEALTHY	15	75	3645	0	728	309	35024	198	1044	14	65
	LOBESIA	0	24	77	402	3747	1001	284	17019	220	1	49
	MILDIU	0	3061	7919	0	13	55	374	673	7894	8	16
	OIDIO	0	0	0	361	129	31	135	164	0	5143	5
	RED_MITE	38	19	476	454	2	1041	483	87	0	41	2927
		ALTICA	BLACK_ROT	BOTRYTIS	EMPOASCA	ERINOSIS	ESCA	HEALTHY	LOBESIA	MILDIU	OIDIO	RED_MITE

Patología Predicha

Figura 18 Matriz de Confusión del modelo DEL en Test

METRICAS DEL MODELO ENTRENADO (Deep Ensemble Learning)

	precision	recall	f1-score
ALTICA	0.872672	0.901364	0.886786
BLACK_ROT	0.684173	0.510350	0.584614
BOTRYTIS	0.408613	0.539925	0.465180
EMPOASCA	0.945377	0.972027	0.958517
ERINOSIS	0.529408	0.638716	0.578947
ESCA	0.885136	0.975604	0.928171
HEALTHY	0.861472	0.851813	0.856615
LOBESIA	0.724059	0.745662	0.734702
MILDIU	0.448395	0.394444	0.419693
OIDIO	0.928675	0.861763	0.893968

Figura 19 Métricas por clase del modelo DEL en Test

Como se puede observar en las métricas desglosadas por clase (Figura 19) y en la diagonal de la Matriz de Confusión (Figura 18), el modelo exhibe un rendimiento excepcional en patologías con cinéticas marcadas. Clases correspondientes a plagas y enfermedades estructurales logran métricas casi perfectas:

- Empoasca (F1: 0,9585)
- Esca (F1: 0,9282)
- Altica (F1: 0,8868)
- Oídio (F1: 0,8940)

En estos casos, el solapamiento de características es mínimo y el modelo clasifica con precisión milimétrica (p. ej., de los 23.165 casos reales de Empoasca, 22.517 son identificados correctamente). Sin embargo, la penalización que arrastra la Exactitud Global hacia el 70,52% recae casi en exclusiva en el bloque de las enfermedades fúngicas dependientes de alta humedad libre:

- Botrytis (F1: 0,4652)
- Mildiu (F1: 0,4197)
- Black Rot (F1: 0,5846)

El análisis de los falsos positivos/negativos en la Figura 18 expone claramente esta confusión intrafúngica:

- De los registros reales de Black Rot, el modelo desvía erróneamente 11.357 predicciones hacia Botrytis.
- En los casos reales de Mildiu, la red se confunde pronosticando Botrytis en 7.919 ocasiones y Black Rot en 3.061.
- Inversamente, de los casos de Botrytis, 4.988 son diagnosticados como Mildiu.

Esta redundancia cruzada constituye la mayor limitación biológica del dataset: al depender las tres especies de ventanas estrechas de humedad foliar continua y temperaturas primaverales análogas, el espacio de características sintético carece de la separabilidad matemática estricta necesaria para que el algoritmo diferencie la cepa exacta basándose únicamente en telemetría climática a escala macro.

7.4 Evaluación Operativa y Visión de Negocio

Si bien la evaluación taxonómica cruda (11 clases) define el límite técnico del ecosistema simulado (F1-Score 0,7259), resulta imperativo alinear el rendimiento algorítmico con el objetivo final de negocio: la reducción del uso de productos fitosanitarios en un 10%.

A nivel agronómico, la toma de decisiones como Sistema de Soporte no requiere de una discriminación microbiológica perfecta si el tratamiento de mitigación es idéntico. La confusión intrafúngica observada (Botrytis vs. Black Rot vs. Mildiu) carece de impacto negativo en campo, dado que las tres patologías activan el mismo protocolo de actuación: la aplicación de fungicidas sistémicos o penetrantes de amplio espectro.

Para evaluar la viabilidad operativa real, se agruparon taxonómicamente las 11 etiologías en 5 Familias de Tratamiento (Sano, Tratamiento Fúngico, Tratamiento Insecticida, Tratamiento Acaricida y Tratamiento Estructural).

Familia Real	Sano (Sin Acción)	35024	793	309	4778	213
	Tratamiento Acaricida	1416	10310	1163	1330	2905
	Tratamiento Estructural	76	171	21515	49	242
	Tratamiento Fúngico	3698	1894	319	85322	3999
	Tratamiento Insecticida	442	3836	1001	652	42917
		Sano (Sin Acción)	Tratamiento Acaricida	Tratamiento Estructural	Tratamiento Fúngico	Tratamiento Insecticida
		Familia Predicha				

Figura 20 Métricas del modelo DEL en Test agrupando por familias de patologías en función del tratamiento

Al proyectar las inferencias del modelo sobre esta dimensionalidad operativa (Figura 20), el ruido estadístico de la redundancia climática desaparece. La nueva matriz consolida un rendimiento superlativo:

- **Precisión en Tratamientos de Alto Coste:** La familia de Tratamiento Fúngico agrupa un total de 85.322 Verdaderos Positivos, demostrando que el modelo detecta el riesgo fúngico global de manera formidable, neutralizando la penalización cruzada observada en las 11 clases.

- **Mantenimiento del Estatus Sano:** El modelo preserva de forma precisa el diagnóstico de Sano (Sin Acción) (35.024 aciertos). Esta robustez en el filtrado de escenarios libres de riesgo es la métrica clave que garantiza la cancelación de tratamientos rutinarios "a ciegas", asegurando matemáticamente el ahorro del 10% en agroquímicos.

En conclusión, bajo la agrupación operativa orientada a la toma de decisiones reales (Figura 20), el sistema experto incrementa drásticamente sus métricas operativas. La arquitectura DEL no solo es un clasificador temporal robusto, sino una herramienta de optimización de recursos agronómicos plenamente viable para un despliegue IoT en producción.

8 Despliegue e Inferencia del Modelo

8.1 Modo de Despliegue

El modelo central, que ensambla tres arquitecturas de redes neuronales complementarias (LSTM, 1D-CNN, y BiGRU), está diseñado para ejecución local mediante el empaquetado de scripts en Python (.py).

La interacción se realiza a través de una **interfaz de línea de comandos (CLI)** accesible mediante el orquestador principal **src/main.py** o utilizando los accesos directos de la carpeta **scripts/**. El sistema se ejecuta como procesos batch bajo demanda o programados (por ejemplo, mediante tareas planificadas tipo CRON o el programador de tareas de Windows), sin necesidad de exponer una API persistente, lo que simplifica su operación y mantenimiento.

8.2 Mecanismo de Inferencia

La lógica de inferencia está encapsulada para garantizar que los datos nuevos sufran exactamente las mismas transformaciones temporales e ingenieriles que los datos utilizados durante el entrenamiento. El flujo de ejecución estándar es el siguiente:

1. **Carga de datos:** Lectura de un fichero origen (CSV o Parquet) con los registros horarios de sensores y aseguramiento de la ventana temporal. Si la serie histórica posee más de 7 días (168 horas) continuadas, el sistema recorta conservando únicamente los 7 últimos días de datos. Si posee menos, replica la primera fila tantas veces como sea necesario al inicio de la

- secuencia (padding por repetición) para mantener el formato matricial esperado.
2. **Ingeniería de características:** Generación de los mismos descriptores que en entrenamiento, calculados de forma determinista para la configuración de la ventana. Específicamente, se extraen descriptores temporales cíclicos (Hora_Sin, Hora_Cos) empleando la fecha. Se calculan las métricas sin requerir lógicamente las variables target de patología y severidad.
 3. **Carga de artefactos de producción:** Lectura desde la carpeta *models/artifacts/* de:
 - **scaler.pkl:** objeto de normalización general con las estadísticas físicas congeladas del entrenamiento.
 - **label_encoder.pkl:** mapeo bidireccional para decodificar numéricamente la afección patológica predicha a una cadena de texto tratable.
 - **M1_LSTM.keras, M2_CNN.keras, M3_BiGRU.keras:** redes neuronales entrenadas listas para la inferencia.
 4. **Predicción (Deep Ensemble Learning):** Aplicación del escalado transformando los datos al formato de tensor 3D de 7 días (168 horas) y pase secuencial de registros por los tres modelos compilados. Finalmente, la integración del ensamblado se realiza mediante Soft Voting, con pesos uniformes ($w_i = 1/3$ por modelo), para la clasificación patológica predominante y Media Aritmética para asentar el grado numérico de severidad.
 5. **Salida:** A partir del diagnóstico arrojado, se consulta internamente una base de conocimiento predefinida asignando de forma automática los regímenes de tratamiento sugeridos (acciones de tipo químico, biológico, biotecnológico, cultural, etc.). Estos se integran al DataFrame final que reporta la confianza predictiva y sus variables determinantes.

El flujo se implementa en la función principal de inferencia *run_inference(input_csv, config, id_serie=None)*, ubicada en **src/predict/predictor.py**, a la que se accede desde la CLI mediante el comando de inferencia.

Comando de inferencia:

```
python -m src.main predict
```

Para sobrescribir la fuente de datos dinámica por defecto, se añaden los argumentos (flags) correspondientes:

```
python -m src.main predict
--input "ruta_nuevo_sensor.parquet"
```

`--id_serie <ID>`

El sistema genera automáticamente el histórico transaccional exportando los resultados al fichero ***inferencia_vid.csv*** en la carpeta *data/predictions/*. Este nombre y la ruta se pueden configurar desde ***config/config.yaml***.

En caso de no encontrar el archivo para predecir (si no encuentra ***data/raw/data_vin_raw.parquet*** por defecto o la ruta al sensor utilizando el flag `--input "ruta"`) se devuelve un mensaje de error y se aborta la ejecución.

Formato de entradas para inferencia:

- **Formato:** Archivo CSV o Parquet.
- **Método de paso:** Argumento `--input` en la línea de comandos (o el archivo de histórico crudo en ***data/raw/data_vin_raw.parquet*** por defecto si este no es forzado desde afuera).
- **Histórico requerido:** Al menos 7 días (168 horas) previos de evaluación por sensor individual, de forma que el sistema pueda constituir los tensores temporales precisos y estabilizar los ciclos sin/cos de temperatura y humedad.

Formato de salidas:

- **Formato:** Archivo CSV.
- **Ubicación:** Carpeta *data/predictions/*.
- **Contenido:** DataFrame que contiene las siguientes columnas:
 - **ID_Serie:** ID de la Serie que corresponde a la predicción, si hay varios y no se especifica el ID que se quiere predecir (con el flag `--id_serie <ID>`) se agrupan todas las series y se predice individualmente cada una de ellas.
 - **Fecha_Evaluacion:** Fecha (*datetime*) en la que se realiza la predicción.
 - **Diagnostico_IA:** Predicción de la patología o estado de la planta en ese instante (predicción de clasificación) como *string*.
 - **Confianza_Clasificacion:** Confianza predictiva del modelo para clasificar dicho estado.
 - **Grado_Severidad:** Predicción del grado de severidad (regresión).
 - **Tratamiento_Recomendado:** Tratamiento Recomendado para la plaga o enfermedad identificada, basada en el diccionario construido a través de la base de conocimientos [2].

9 Reproducibilidad y Evidencias

9.1 Inventario de Artefactos y Entorno

Para garantizar la reproducibilidad del modelo central, se entregan los siguientes elementos críticos dentro de sus respectivos directorios (no deben modificarse manualmente):

- Modelos de producción y Pesos – ubicados en **models/artifacts/**:
 - **M1_LSTM.keras, M2_CNN.keras, M3_BiGRU.keras**: pesos y topologías binarias ya computadas.
 - **scaler.pkl**: objeto StandardScaler congelando los centros estadísticos de preprocesamiento.
 - **label_encoder.pkl**: empaquetado del diccionario de las clases patológicas y plagas a su estado identificativo.
- **Definición del entorno**
 - **requirements.txt**: lista estricta de dependencias y versiones (tensorflow, pandas, scikit-learn, joblib, numpy, etc.) necesarias para asegurar la replicabilidad numérica.

9.2 Pasos para Reproducir el Ciclo Completo

Paso 1 – Configuración del entorno e Instalación de dependencias

1. Crear entorno virtual: **python -m venv venv**
2. Activar entorno (Windows): **venv\Scripts\activate**
3. Activar entorno (Linux/Mac): **source venv/bin/activate**
4. Instalar librerías: **pip install -r requirements.txt**

Paso 2 – Obtención de datos (ETL y Simulador)

Aunque los datos crudos y los splits se encuentran en un contenedor externo, si no se dispone del dataset original masivo exportable originado por sensores, debe ejecutarse el código del simulador de infraestructura para recrear artificialmente el registro base y guardarlo en **data/raw/data_vin_raw.parquet**. Con los siguientes comandos se obtienen los datos meteorológicos históricos mediante la [API Open-Meteo](#) (toda la lógica de descarga, zonas, fechas y modelo se pueden editar desde el script para cubrir las necesidades del sistema) con los que se simulan las series temporales de enfermedades y plagas (salida en la carpeta **data/clima_real/**), el resultado de limpiarlos y procesarlos para evitar nulos y datos faltantes (salida en la carpeta **data/clima_real/clean/**) y el conjunto de

datos sintético en crudo para recrear todo el pipeline del proyecto (salida en la carpeta **data/raw/**).

Para recrear todo el flujo de obtención de datos se deben seguir los siguientes pasos:

1. Descarga de datos climáticos históricos de 5 cultivos diferentes de CyL mediante la API de Open-Meteo:

```
python scripts/data/descarga_clima_historico.py
```

2. Limpieza de estos datos por si hubiera datos faltantes o nulos:

```
python scripts/data/data_cleaner.py
```

3. Simulación del desarrollo de plagas o enfermedades para crear el dataset sintético definitivo:

```
python scripts/data/vid_simulator.py
```

Paso 3 - Obtención de datos (Procesamiento)

Tras garantizar que los datos crudos residen en la carpeta correspondiente, se estructuran limpiamente inyectando la ingeniería de variables temporales, grados acumulados y horas de humedad foliar, generando el dataset base **data/processed/data_vin_processed.parquet**:

```
python -m src.main data_processing
```

Paso 4 - Ejecución del pipeline de Entrenamiento y Evaluación

Este comando particiona los bloques, aplica escalados, genera las secuencias temporales de 7 días (168 horas) y entrena los tres modelos. Con el flag **--metrics**, realiza la inferencia sobre el conjunto de test para generar reportes sin fuga de datos:

```
python -m src.main train --metrics
```

Paso 5 (OPCIONAL) - Extracción de estadísticas

Para extraer las estadísticas que describen el sistema:

```
python -m src.main get_stats
```

(Generará en formato *markdown* una descripción del modelo, un reporte estadístico de las variables exportado a **data/processed/estadisticas.csv**, una descripción de cada variable en **feature_descriptions.csv** y una evaluación del modelo DEL por terminal).

Paso 6 - Ejecución de Inferencia

Para realizar predicciones en producción sobre datos nuevos procedentes de los sensores:

```
python -m src.main predict
```

Donde se da la posibilidad de utilizar los siguientes argumentos (flags) opcionales:

- **--input data/raw/datos_nuevos.parquet:** Ruta del dataset de entrada (sobrescribe el de config). Por defecto se utiliza **raw/data_vin_raw.parquet**.
- **--id_serie <ID>:** Filtra y predice sobre una única serie temporal en específico. En caso de no especificar el ciclo, se realizará una predicción de cada uno de los identificadores existentes en el conjunto de datos.

9.3 Trazabilidad del Código

El código completo está modularizado bajo el ecosistema madre *src/*, envuelto a nivel global por los disparadores de *scripts/*, con los siguientes puntos de entrada para auditoría:

- **Orquestador principal:** *src/main.py*, responsable de enlazar los módulos según los argumentos recibidos por la CLI.
- **Lógica de entrenamiento:** Ubicada y clasificada exhaustivamente en *src/training/*, donde se implementan los constructores de arquitecturas neuronales, métricas propias y callbacks de memoria.
- **Lógica de inferencia:** Ubicada en *src/predict/predictor.py*, responsable de la carga de tensores simulados, artefactos seriales y acoplamiento directo del módulo de base de conocimientos expertos.
- **Lógica de procesamiento:** Módulo funcional de *src/data_processing/*, enlazando funciones auxiliares de depuración y formateo.

9.4 Evidencias y Resultados

Tras completar la validación sobre sus particiones de retención, paralelamente el sistema autogenera evidencias de monitorización:

- **models/metrics/<Modelo>/:** Reportes en formato .csv (métricas de clasificación y regresión por submodelo y ensamble) y matrices de confusión visuales en formato .png. Además, se obtiene el historial del aprendizaje de cada uno de los modelos en formato csv con nombre **history_<Modelo>.csv** para trazar las curvas de aprendizaje durante las *Epochs*.
- **data/splits/:** Conserva los conjuntos de datos inmutables train.parquet, val.parquet y test.parquet.
- **data/predictions/inferencia_vid.csv:** Archivo final de test funcional de muestra fila a fila referenciando si la decisión tomada por los módulos de inferencia, confiabilidad y enrutamiento en forma de aviso se completaron de modo idóneo sobre un caso puramente real ciego.

10 Limitaciones, Riesgos y Conclusiones

Para garantizar una transferencia segura del sistema de aprendizaje profundo en ensamble (Deep Ensemble Learning) a entornos de explotación agrícola reales, se deben auditar críticamente las restricciones metodológicas, técnicas y operativas que definen el estado actual del Gemelo Digital.

10.1 Desafíos y Limitaciones del Conjunto de Datos

- **Naturaleza Híbrida del Espacio Dimensional:** La base climática del sistema se nutre de series meteorológicas históricas genuinas. Esto confiere al modelo una robustez estructural óptima frente a ciclos termodinámicos y anomalías meteorológicas reales. Sin embargo, ante la inexistencia de repositorios públicos masivos y rigurosamente etiquetados sobre la severidad de las infecciones, el comportamiento biológico de los patógenos y la consecuente emisión de biomarcadores gaseosos (VOC_ppb y CO2_ppm) responden a ecuaciones paramétricas teóricas. Por tanto, aunque el input climático es veraz, la correlación inmunológica subyacente es una aproximación computacional, lo que constituye la principal limitación del sistema.

- **Modelado del Ruido Físico en Producción:** El dataset expone al ensamble al ruido meteorológico real de la región. No obstante, el despliegue físico en una infraestructura IIoT real implicará lidiar con perturbaciones físicas adicionales e inherentes al hardware de campo: descalibraciones progresivas por deriva galvánica de los sensores, pérdidas de paquetes de red en la transmisión inalámbrica o degradación química de las sondas edáficas. Estas variables de error instrumental no se encuentran completamente modeladas en la matriz de entrenamiento estática actual.
- **Acotación Topológica de Clases:** El clasificador está restringido rígidamente al catálogo de las 11 etiologías fitosanitarias parametrizadas durante la modelización. La aparición de patologías exóticas, mutaciones biológicas, plagas invasoras o eventos de estrés hídrico extremo no catalogados previamente será indetectable por el modelo o, de forma crítica, inducirá falsos positivos al forzar al vector Softmax a clasificar la anomalía dentro de las fronteras de decisión conocidas.
- **Restricciones Computacionales en el Ajuste:** El volumen neto de las series temporales multivariantes se calibró para equilibrar de forma óptima la viabilidad de cómputo y la representatividad estadística. El procesamiento de tensores tridimensionales masivos con granularidad horaria a escala regional incrementa la latencia de cómputo a escala de días, limitando la exhaustividad del barrido estocástico para la búsqueda de hiperparámetros (Hyperparameter Tuning) y la validación cruzada extendida.

10.2 Implicaciones Operativas en la Toma de Decisiones

- **Estatus de Apoyo Clínico:** La Inteligencia Artificial desarrollada debe catalogarse exclusivamente como un Sistema de Soporte a la Decisión y una herramienta de cribado o alerta temprana para priorizar e inspeccionar parcelas en riesgo. Bajo ningún concepto metodológico su dictamen pretende sustituir el juicio empírico, la experiencia fenológica y la supervisión técnica de ingenieros agrónomos, peritos o viticultores.
- **Protocolo de Validación Humana Obligatoria:** Cualquier alerta emitida por el ensamble que sugiera intervenciones químicas severas (como tratamientos erradicantes de alto impacto), así como aquellas predicciones multitarea que registren una dispersión elevada en la entropía del Soft Voting (baja confianza diagnóstica), exigen una auditoría física obligatoria

sobre el terreno por parte del operador antes de activar cualquier acción fitosanitaria.

- **Modularidad de la Instrumentación en Producción:** La arquitectura del sistema ha sido concebida bajo un diseño altamente flexible para adaptarse a las capacidades de infraestructura real de cada explotación. En caso de que un viñedo no disponga de la totalidad de la red de sensores estipulada (por restricciones logísticas o presupuestarias), el sistema puede operar bajo configuraciones restrictivas. Bastará con excluir la variable ausente en el archivo de parametrización (**config/config.yaml**) y ejecutar de nuevo el pipeline de entrenamiento; el ensamble neuronal reajustará sus tensores y matrices de pesos para extraer el máximo rendimiento exclusivamente de la telemetría disponible. Esta flexibilidad dota al proyecto de una excelente escalabilidad, democratizando su despliegue en viñedos con recursos IoT más básicos. No obstante, desde el punto de vista operativo, debe asumirse el compromiso técnico (trade-off) subyacente: la supresión de predictores físicos clave penalizará de forma inherente la capacidad de detección temprana del modelo, tal y como se demostró empíricamente en el estudio restrictivo reflejado en la Tabla 9.

10.3 Comportamiento Esperado en Despliegue

- **Mitigación del Cambio de Dominio Climático:** Debido a la inyección de series históricas multivariantes reales en el pipeline de ETL, el riesgo de colapso predictivo por cambio de dominio (Domain Shift) climático se reduce al mínimo. Las redes neuronales profundas del ensamble han estabilizado sus matrices de pesos frente a las distribuciones físicas veraces de Castilla y León, por lo que la lectura de estaciones agroclimáticas en campo real no supondrá una ruptura en las densidades de entrada del modelo.
- **Brecha de Transferencia Biológica (Sim-to-Real Gap):** A pesar de la robustez climática, se anticipa una degradación controlada de las métricas nominales del test hold-out (Accuracy y F1-Score) al operar sobre viñedos vivos. La varianza microclimática a nivel de haz y envés de la hoja, la resistencia genética diferencial de las variedades de vid, la inercia inmunitaria de la planta y el impacto residual de tratamientos profilácticos previos configuran un entorno biológico intrínsecamente más caótico que el modelado por el simulador computacional.

10.4 Escalabilidad y Líneas de Trabajo Futuro

- **Sustitución por Ground Truth:** La prioridad inminente para la evolución del Gemelo Digital no radica en incrementar la profundidad de los hiperparámetros de las redes neuronales, sino en conectar la CLI del predictor a una red física IIoT desplegada en parcelas comerciales. El objetivo es iniciar la captura masiva de diagnósticos empíricos de campo para sustituir gradualmente los objetivos sintéticos por etiquetas biológicas reales, cerrando de forma definitiva la brecha Sim-to-Real.
- **Agnosticismo de la Arquitectura del Sistema:** La ingeniería desarrollada (estructuración de ventanas temporales 3D, proyecciones ciclotómicas mediante variables sinusoidales y la topología DEL multitarea) es completamente agnóstica al tipo de cultivo. El pipeline analítico es altamente escalable y podría extrapolarse de forma directa a plantaciones hortícolas bajo plástico, olivares o frutales de hueso, requiriendo únicamente la alimentación de un nuevo dataset que codifique las dinámicas mecanicistas de las plagas del ecosistema objetivo. Del mismo modo, para el caso de uso vitivinícola, se podría ampliar o reducir el número de patologías a detectar por este sistema desde el archivo de configuración (config/config.yaml).

10.5 Conclusiones Generales

El sistema ensamblado ha evidenciado un elevado potencial arquitectónico como motor predictivo para la sanidad vegetal de precisión, superando con éxito el desafío metodológico de integrar y procesar telemetría climática histórica veraz sobre tensores tridimensionales recurrentes. Si bien la eficacia fitopatológica operativa de la solución queda supeditada a una futura validación cruzada con diagnósticos empíricos de campo, las predicciones probabilísticas de la arquitectura de Ensamble Profundo (DEL) han trascendido la fase de concepto teórico, consolidándose como un prototipo funcional avanzado.

La integración simétrica de un bloque convolucional (1D-CNN) y dos bloques recurrentes de memoria (LSTM y BiGRU) ha demostrado una capacidad sobresaliente para amortiguar la varianza estructural y aislar las firmas morfológicas de los biomarcadores gaseosos. Al proyectar las inferencias taxonómicas sobre familias funcionales de tratamiento agronómico, el modelo no solo disuelve el solapamiento agrometeorológico de los patógenos fúngicos, sino que satisface los umbrales de éxito exigidos, alcanzando un F1-Score Macro

operativo de 0,8469 y un ajuste analógico del grado de severidad del 89,03% ($R^2 = 0,8903$).

En definitiva, el sistema proporciona una base analítica robusta, madura y altamente escalable para su integración en plataformas de viticultura inteligente, operando como un pilar fundamental para optimizar la rentabilidad de las explotaciones y garantizar, de manera matemática y segura, la reducción del 10% en la huella química de los tratamientos fitosanitarios.

Referencias

- [1] A. K. Mahlein, "Plant Disease Detection by Imaging Sensors—Parallels and Specific Demands for Precision Agriculture and Plant Phenotyping," *Plant Disease*, vol. 100, no. 2, pp. 241–251, 2016, doi: 10.1094/PDIS-03-15-0340-FE.
- [2] A. L. Espadas, *Plagas y enfermedades de la vid en la Región de Murcia*. Murcia: Consejería de Agricultura y Agua, Servicio de Sanidad Vegetal, 2008.
- [3] R. G. Allen, L. S. Pereira, D. Raes, y M. Smith, Crop evapotranspiration - Guidelines for computing crop water requirements, FAO Irrigation and Drainage Paper 56. Roma, Italia: FAO, 1998. Disponible en: <https://www.fao.org/3/x0490e/x0490e00.htm>
- [4] S. Manabe, "Climate and the ocean circulation: I. The atmospheric circulation and the hydrology of the earth's surface," *Monthly Weather Review*, vol. 97, no. 11, pp. 739–774, 1969. doi: 10.14601/Phytopathol_Mediterr-1847
- [5] K. E. Saxton y W. J. Rawls, "Soil water characteristic estimates by texture and organic matter for hydrologic solutions," *Soil Science Society of America Journal*, vol. 70, no. 5, pp. 1569–1578, 2006, doi: 10.2136/sssaj2005.0117.
- [6] Davidson, E., Belk, E. and Boone, R.D. (1998) Soil Water Content and Temperature as Independent or Confounded Factors Controlling Soil Respiration in a Temperate Mixed Hardwood Forest. *Global Change Biology*, 4, 217-227.
<https://doi.org/10.1046/j.1365-2486.1998.00128.x>
- [7] A. Guenther et al., "A global model of natural volatile organic compound emissions," *Journal of Geophysical Research: Atmospheres*, vol. 100, no. D5, pp. 8873–8892, 1995.
- [8] R. A. Spotts, "Effect of leaf wetness duration and temperature on the infectivity of *Guignardia bidwellii* on grape leaves," *Phytopathology*, vol. 67, no. 11, pp. 1378–1381, 1977, doi: 10.1094/Phyto-67-1378.

- [9] J. C. Broome, J. T. English, J. J. Marois, B. A. Latorre, y J. C. Avilés, "Development of an infection model for Botrytis bunch rot of grapes based on wetness duration and temperature," *Phytopathology*, vol. 85, pp. 97–102, 1995. doi: 10.1094/Phyto-85-97.
- [10] G. Fedele, F. Bove, E. González-Domínguez, y V. Rossi, "A generic model accounting for the interactions among pathogens, host plants, biocontrol agents, and the environment, with parametrization for *Botrytis cinerea* on grapevines," *Agronomy*, vol. 10, no. 2, p. 222, 2020, doi: 10.3390/agronomy10020222.
- [11] G. Barrios y J. Reyes, "Modelización del mildiu de la vid", *Phytoma*, núm. 164, pp. 1-9, dic. 2004.
- [12] V. Rossi, T. Caffi, S. Giosuè, and R. Bugiani, "A mechanistic model simulating primary infections of downy mildew in grapevine," *Ecological Modelling*, vol. 212, no. 3–4, pp. 480–491, Apr. 2008, doi: 10.1016/j.ecolmodel.2007.10.046.
- [13] T. Caffi, V. Rossi, S. E. Legler, y R. Bugiani, "A mechanistic model simulating ascospore infections by *Erysiphe necator*, the powdery mildew fungus of grapevine," *Plant Pathology*, vol. 60, no. 3, pp. 522–531, 2011, doi: 10.1111/j.1365-3059.2010.02395.x.
- [14] D. M. Gadoury, R. C. Seem, A. Ficke, and W. F. Wilcox, "Ontogenic Resistance to Powdery Mildew in Grape Berries," *Phytopathology*®, vol. 93, no. 5, pp. 547–555, May 2003, doi: 10.1094/PHYTO.2003.93.5.547.
- [15] C. S. Thomas, W. D. Gubler, y G. Leavitt, "Field testing of a powdery mildew disease forecast model on grapes in California," *Phytopathology*, vol. 84, no. 10, p. 1070, 1994 (abstr.).
- [16] L. Mugnai, A. Graniti, and G. Surico, "Esca (Black Measles) and Brown Wood-Streaking: Two Old and Elusive Diseases of Grapevines," *Plant Disease*, vol. 83, no. 5, pp. 404–418, May 1999, doi: 10.1094/pdis.1999.83.5.404.
- [17] G. Surico, L. Mugnai, y G. Marchi, "Older and more recent observations on esca: A critical overview," *Phytopathologia Mediterranea*, vol. 45, pp. S68–S86, Abr. 2006, doi: 10.14601/Phytopathol_Mediterr-1847.
- [18] G. Marchi, F. Peduto, L. Mugnai, S. Di Marco, F. Calzarano, y G. Surico, "Some observations on the relationship of manifest and hidden esca to rainfall," *Phytopathologia Mediterranea*, vol. 45, pp. S117–S126, 2006.
- [19] C. A. Flores y G. Azín, "Modelos lineales para pronosticar la actividad de vuelo de *Lobesia botrana* (Denis & Schifferrmüller) (Lepidoptera: Tortricidae) en la localidad de Russel, Mendoza, Argentina," en XVI Reunión Argentina y VIII Latinoamericana de Agrometeorología, Puerto Madryn, Argentina, 2016.

- [20] A. Tobes Velasco, "Procedimiento del seguimiento fenológico y de la curva de vuelo de *Lobesia botrana* en diferentes viñedos de la D.O. Ribera del Duero," Consejo Regulador de la Denominación de Origen Ribera del Duero, Servicio de Experimentación y Ensayo, Roa, España, Informe Técnico, May. 2019.
- [21] G. Gilioli, S. Pasquali, and E. Marchesini, "A modelling framework for pest population dynamics and management: An application to the grape berry moth," *Ecological Modelling*, vol. 320, pp. 348–357, Jan. 2016, doi: 10.1016/j.ecolmodel.2015.10.018.
- [22] K. Kolkman, S. Blümel, and J. Eitzinger, "Predicting the first seasonal occurrence of *Lobesia botrana* and *Eupoecilia ambiguella* in Austria using new multiple linear regression models", *OENO One*, vol. 59, no. 3, Jul. 2025, doi: <https://doi.org/10.20870/oeno-one.2025.59.3.8269>.
- [23] J.-F. Briere, P. Pracros, A.-Y. Le Roux, and J.-S. Pierre, "A Novel Rate Model of Temperature-Dependent Development for Arthropods," *Environmental Entomology*, vol. 28, no. 1, pp. 22–29, Feb. 1999, doi: 10.1093/ee/28.1.22.
- [24] C. Duso y E. Vettorazzo, "Mite population dynamics on different grape varieties with or without phytoseiids released (*Acari: Phytoseiidae*)," *Exp. Appl. Acarol.*, vol. 23, pp. 741–763, 1999, doi: 10.1023/A:1006297225577.
- [25] D. Decante and M. Van Helden, "Spatial and temporal distribution of *Empoasca vitis* within a vineyard," *Agricultural and Forest Entomology*, vol. 10, no. 2, pp. 111–118, Apr. 2008, doi: 10.1111/j.1461-9563.2008.00366.x.
- [26] M. C. Stavrinides, J. R. Lara, and N. J. Mills, "Comparative influence of temperature on development and biological control of two common vineyard pests (*Acari: Tetranychidae*)," *Biological Control*, vol. 55, no. 2, pp. 126–131, Nov. 2010, doi: 10.1016/j.biocontrol.2010.07.006.
- [27] A. Kamilaris y F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey", *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018, doi: 10.1016/j.compag.2018.02.016.
- [28] F. Martinelli *et al.*, "Advanced methods of plant disease detection. A review," *Agronomy for Sustainable Development*, vol. 35, no. 1, pp. 1–25, 2015, doi: 10.1007/s13593-014-0246-1.
- [29] G. Fenu y F. M. Mallocci, "Forecasting plant and crop disease: An explorative study on current algorithms," *Big Data and Cognitive Computing*, vol. 5, no. 1, p. 2, 2021, doi: 10.3390/bdcc5010002.
- [30] M. Hnatiuc, S. Ghita, D. Alpetri, A. Ranca, V. Artem, I. Dina, M. Cosma, and M. Abed Mohammed, "Intelligent Grapevine Disease Detection Using IoT Sensor

Network," Bioengineering, vol. 10, no. 9, p. 1021, 2023, doi: 10.3390/bioengineering10091021.

